Paarth Maisuria

ITDSA

Assignment

Due Date: 02 July

Eduvos

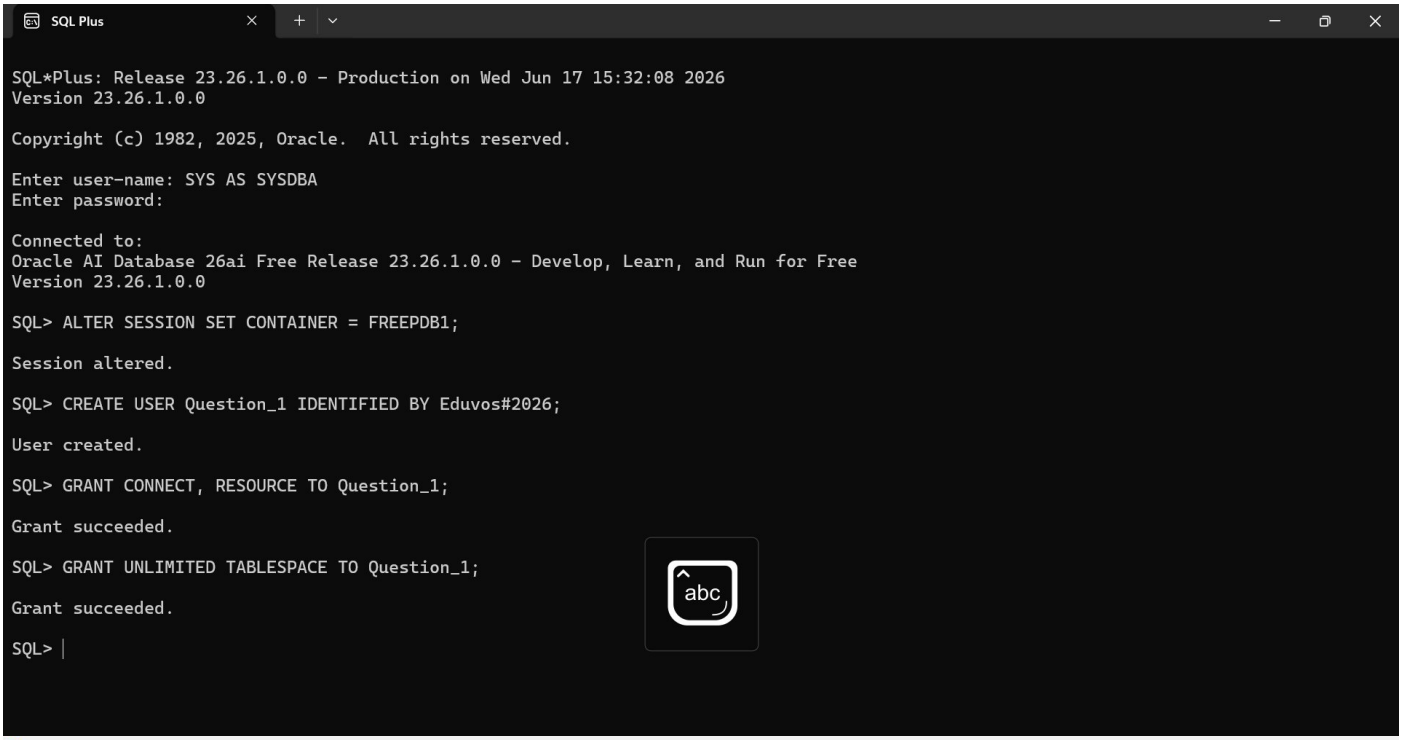
EDUV5606518

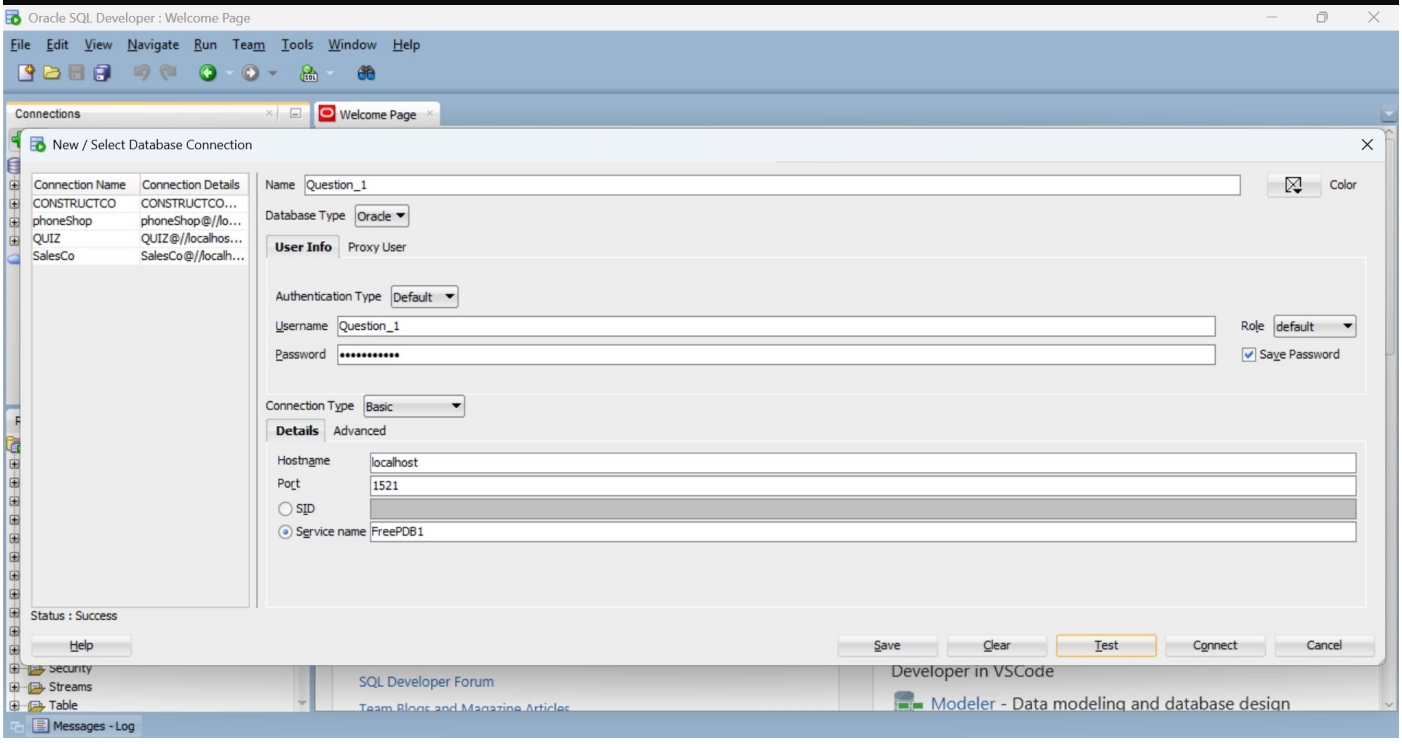


Eduvos

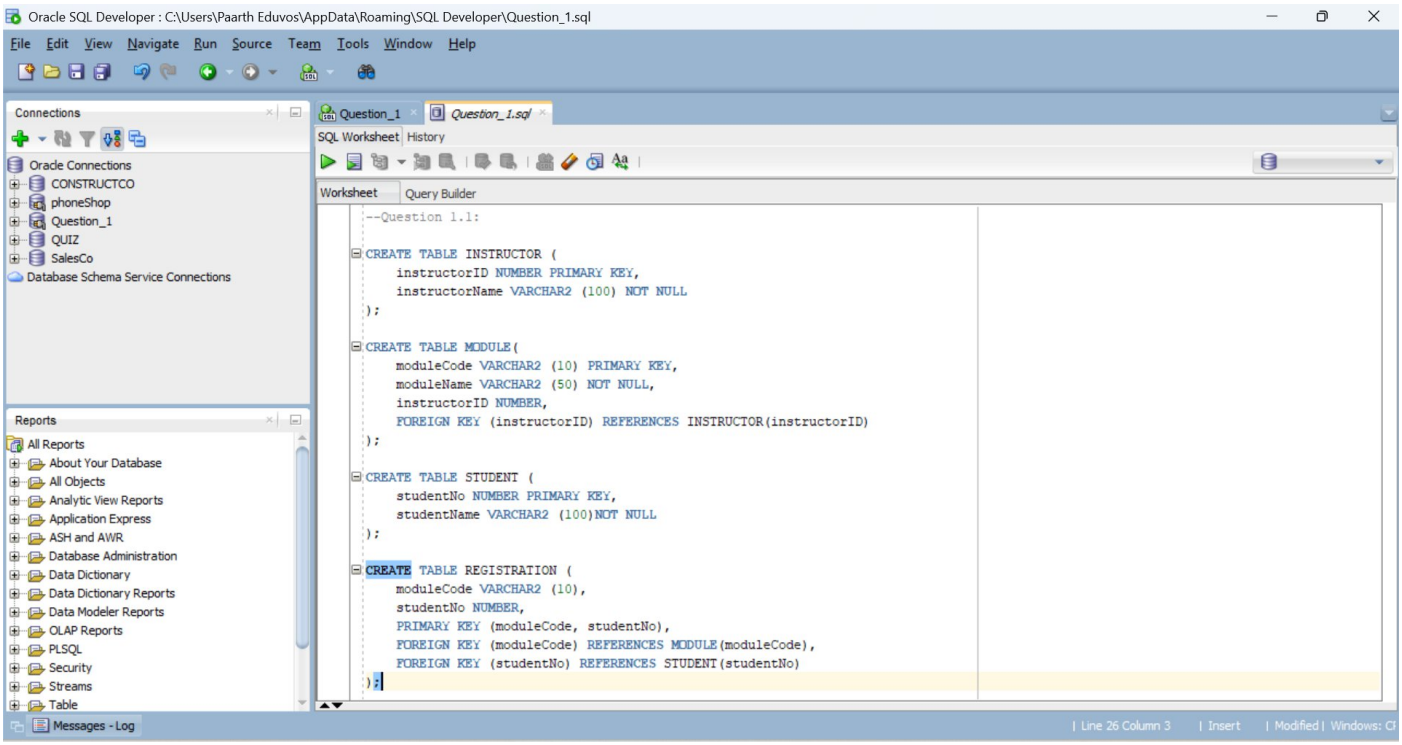
Your Education. Your Future.

Deliverable 1 / Question 1:





1.1)



--Question 1.1:

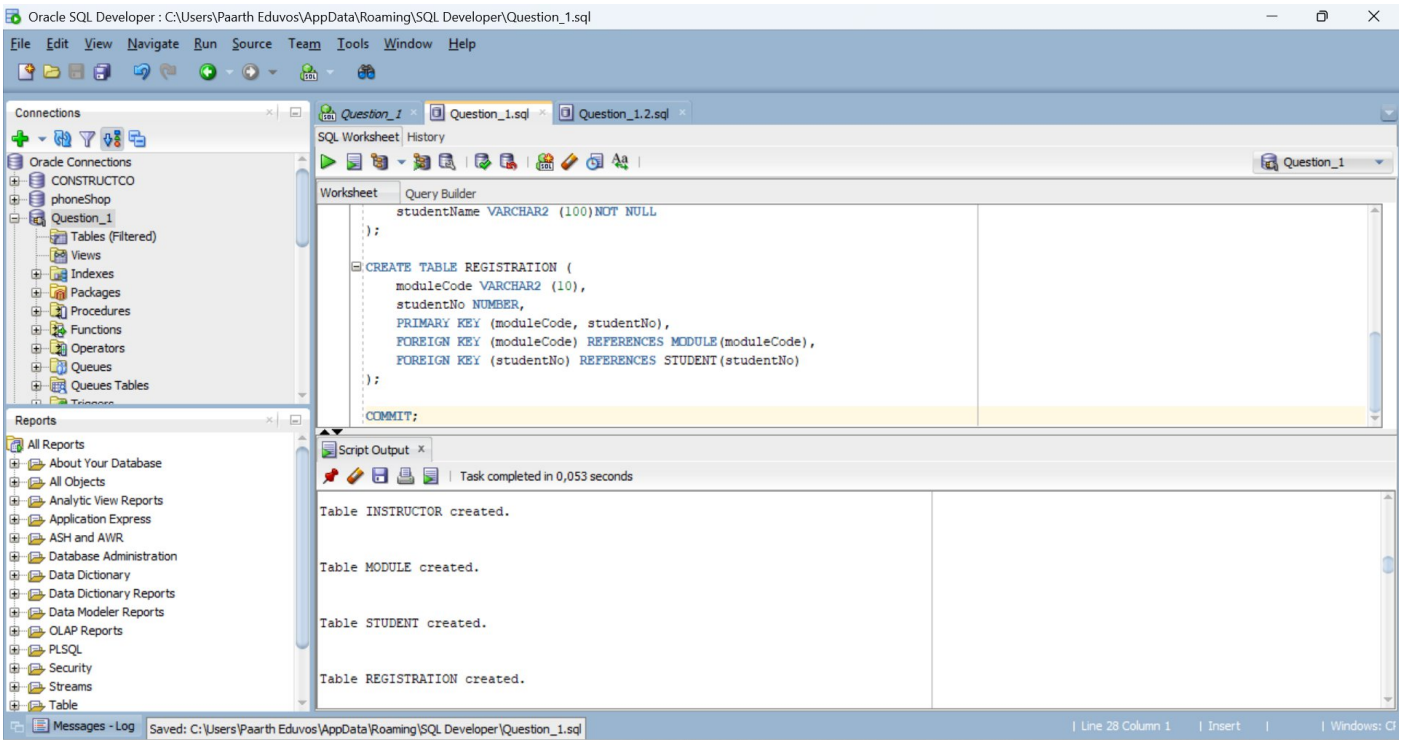
```
CREATE TABLE INSTRUCTOR (
    instructorID NUMBER PRIMARY KEY,
    instructorName VARCHAR2 (100) NOT NULL
);

CREATE TABLE MODULE(
    moduleCode VARCHAR2 (10) PRIMARY KEY,
    moduleName VARCHAR2 (50) NOT NULL,
    instructorID NUMBER,
    FOREIGN KEY (instructorID) REFERENCES INSTRUCTOR(instructorID)
);

CREATE TABLE STUDENT (
    studentNo NUMBER PRIMARY KEY,
    studentName VARCHAR2 (100)NOT NULL
);
```

```
CREATE TABLE REGISTRATION (
    moduleCode VARCHAR2 (10),
    studentNo NUMBER,
    PRIMARY KEY (moduleCode, studentNo),
    FOREIGN KEY (moduleCode) REFERENCES MODULE(moduleCode),
    FOREIGN KEY (studentNo) REFERENCES STUDENT(studentNo)
);

COMMIT;
```



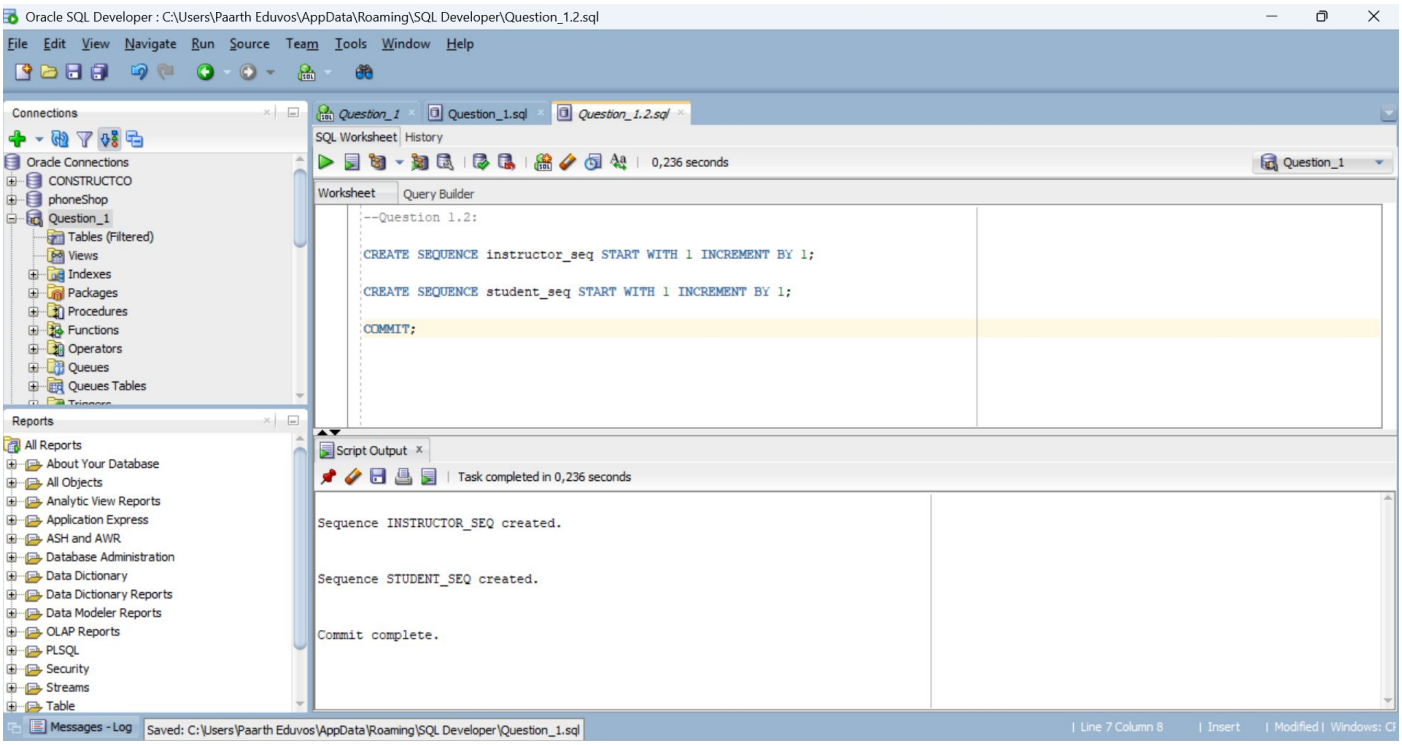
1.2)

--Question 1.2:

```
CREATE SEQUENCE instructor_seq START WITH 1 INCREMENT BY 1;

CREATE SEQUENCE student_seq START WITH 1 INCREMENT BY 1;
```

COMMIT;



1.3)

--Question 1.3:

INSERT INTO INSTRUCTOR (instructorID, instructorName) VALUES (instructor_seq.NEXTVAL, 'Mr Craig Stephens');

INSERT INTO INSTRUCTOR (instructorID, instructorName) VALUES (instructor_seq.NEXTVAL, 'Ms Garcia Lopez');

INSERT INTO MODULE (moduleCode, moduleName, instructorID) VALUES ('ITDSA', 'Database Systems', 1);

INSERT INTO MODULE (moduleCode, moduleName, instructorID) VALUES ('ITAPA', 'Python Programming', 2);

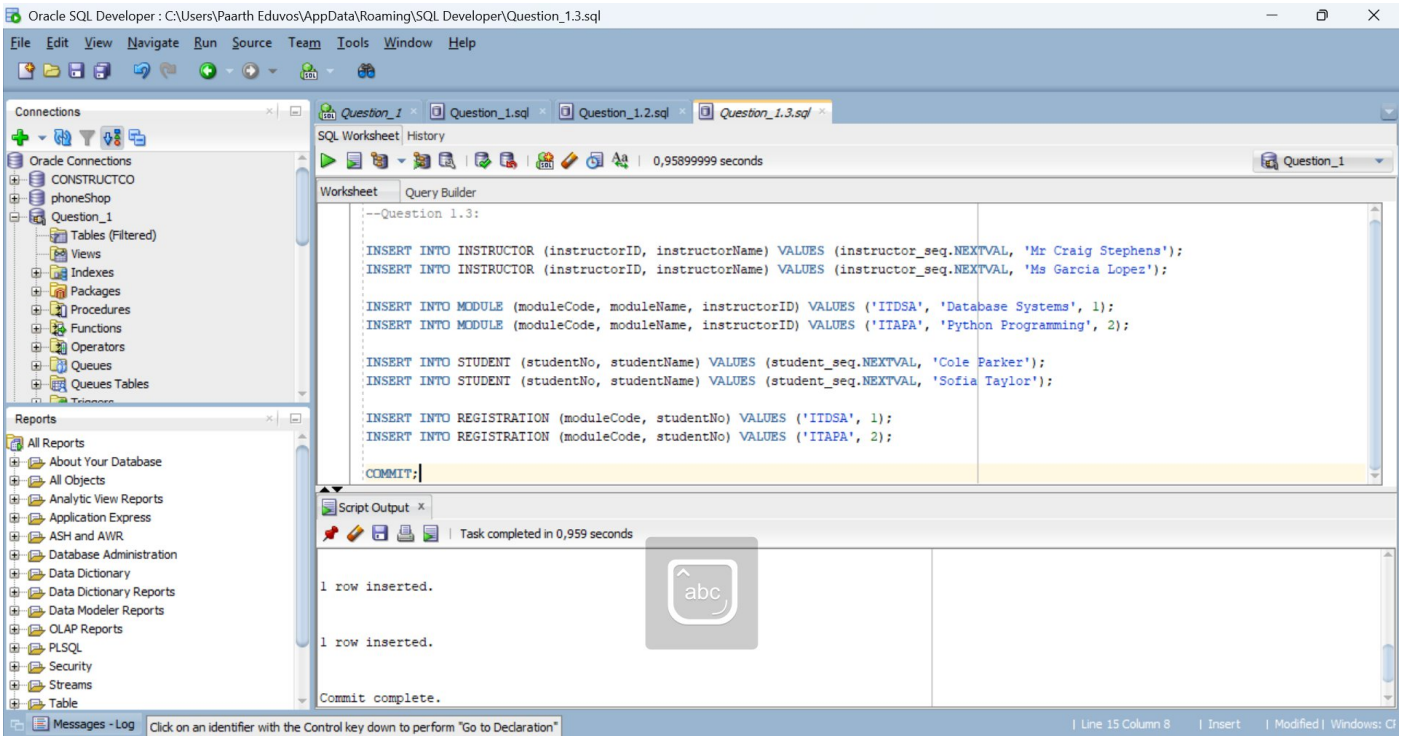
INSERT INTO STUDENT (studentNo, studentName) VALUES (student_seq.NEXTVAL, 'Cole Parker');

INSERT INTO STUDENT (studentNo, studentName) VALUES (student_seq.NEXTVAL, 'Sofia Taylor');

INSERT INTO REGISTRATION (moduleCode, studentNo) VALUES ('ITDSA', 1);

INSERT INTO REGISTRATION (moduleCode, studentNo) VALUES ('ITAPA', 2);

COMMIT;

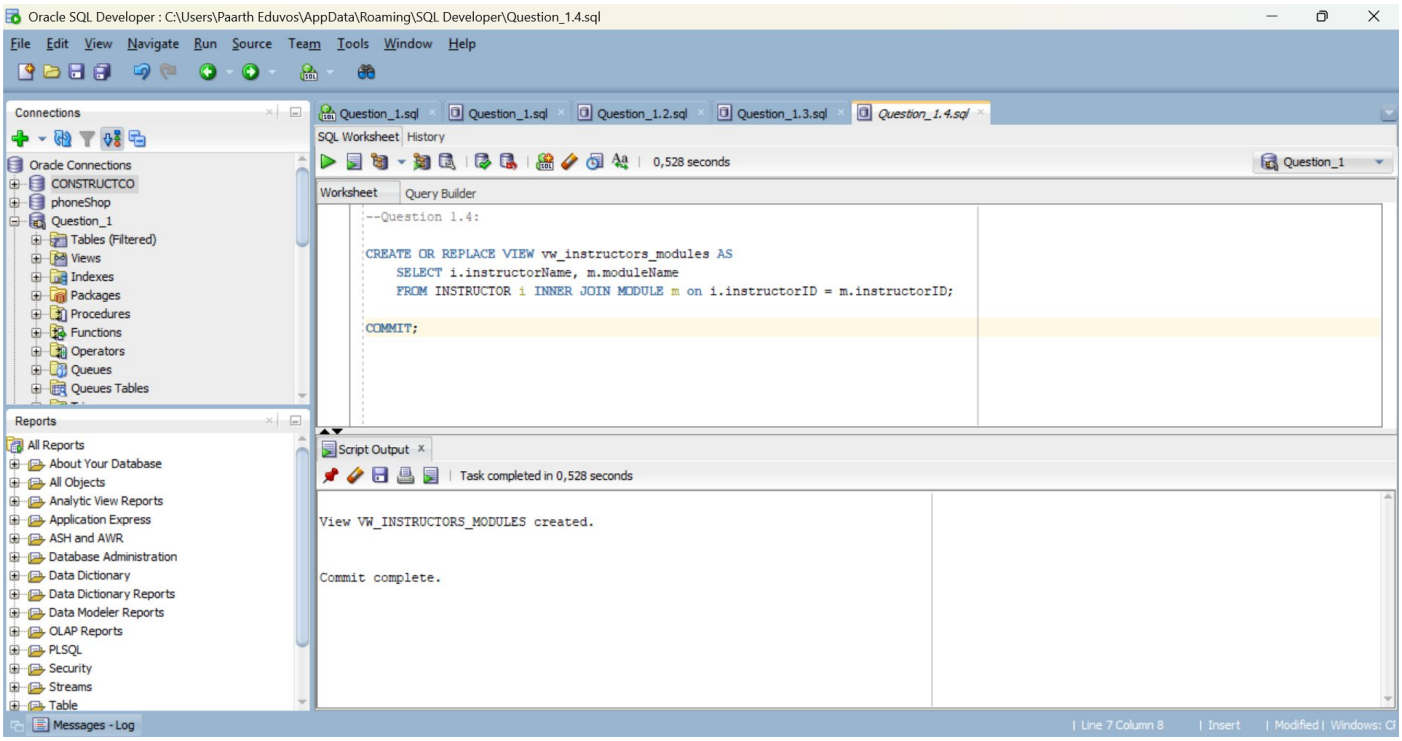


1.4)

--Question 1.4:

```
CREATE OR REPLACE VIEW vw_instructors_modules AS
    SELECT i.instructorName, m.moduleName
    FROM INSTRUCTOR i INNER JOIN MODULE m on i.instructorID = m.instructorID;

COMMIT;
```



1.5)

--Question 1.5:

```
CREATE OR REPLACE PROCEDURE registers_student_for_module(p_studentNo IN NUMBER,
                                                         p_moduleCode IN VARCHAR2) AS

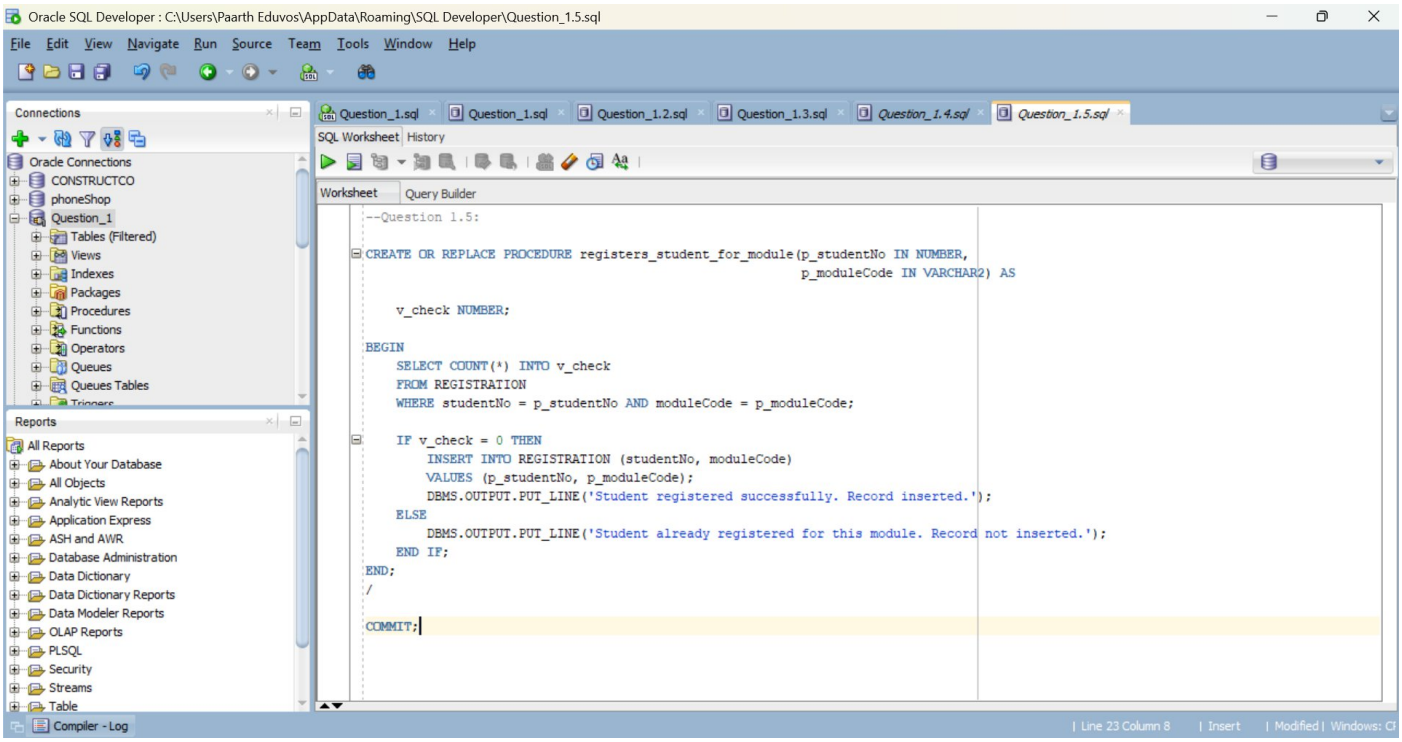
    v_check NUMBER;

BEGIN
    SELECT COUNT(*) INTO v_check
    FROM REGISTRATION
    WHERE studentNo = p_studentNo AND moduleCode = p_moduleCode;

    IF v_check = 0 THEN
        INSERT INTO REGISTRATION (studentNo, moduleCode)
        VALUES (p_studentNo, p_moduleCode);
        DBMS.OUTPUT.PUT_LINE('Student registered successfully. Record inserted.');
    ELSE
        DBMS.OUTPUT.PUT_LINE('Student already registered for this module. Record not
inserted.');
```

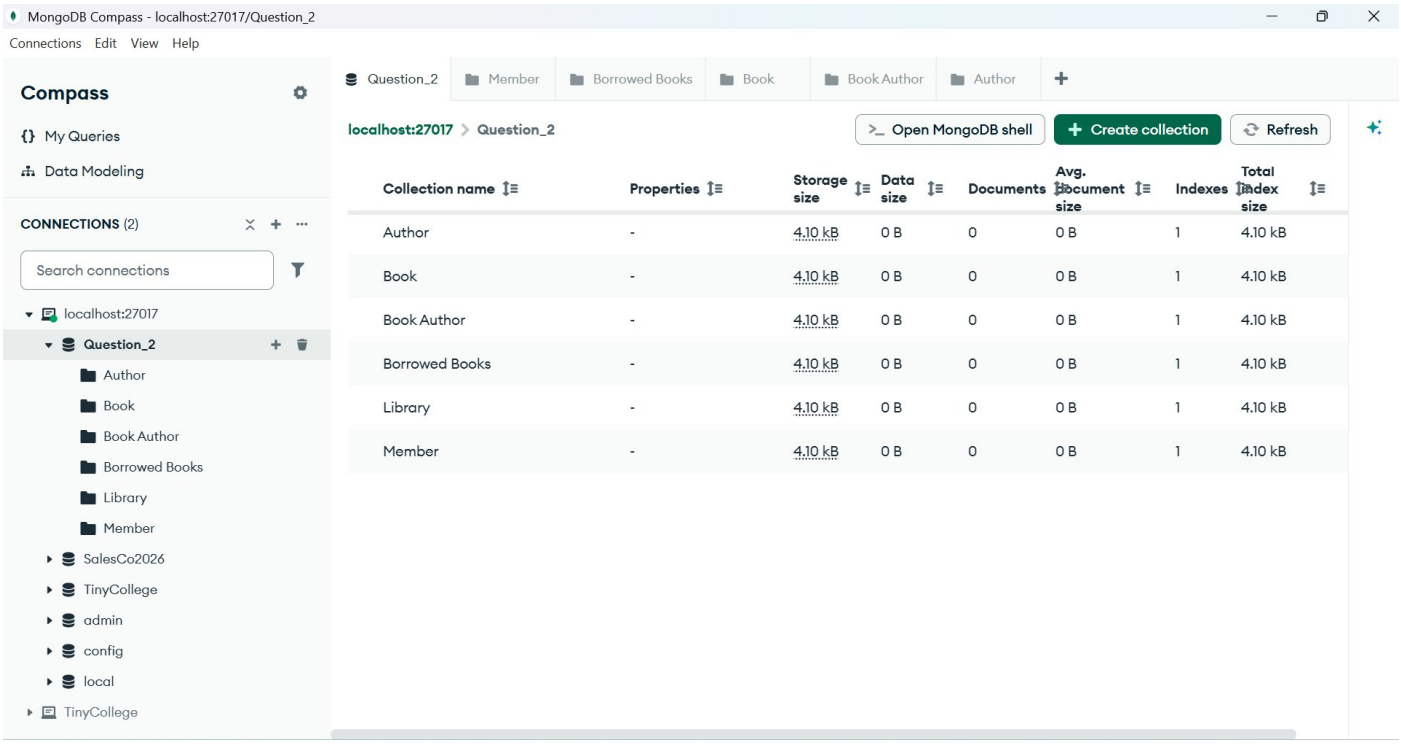
```
    END IF;
END;
/

COMMIT;
```



Deliverable 2 / Question 2:

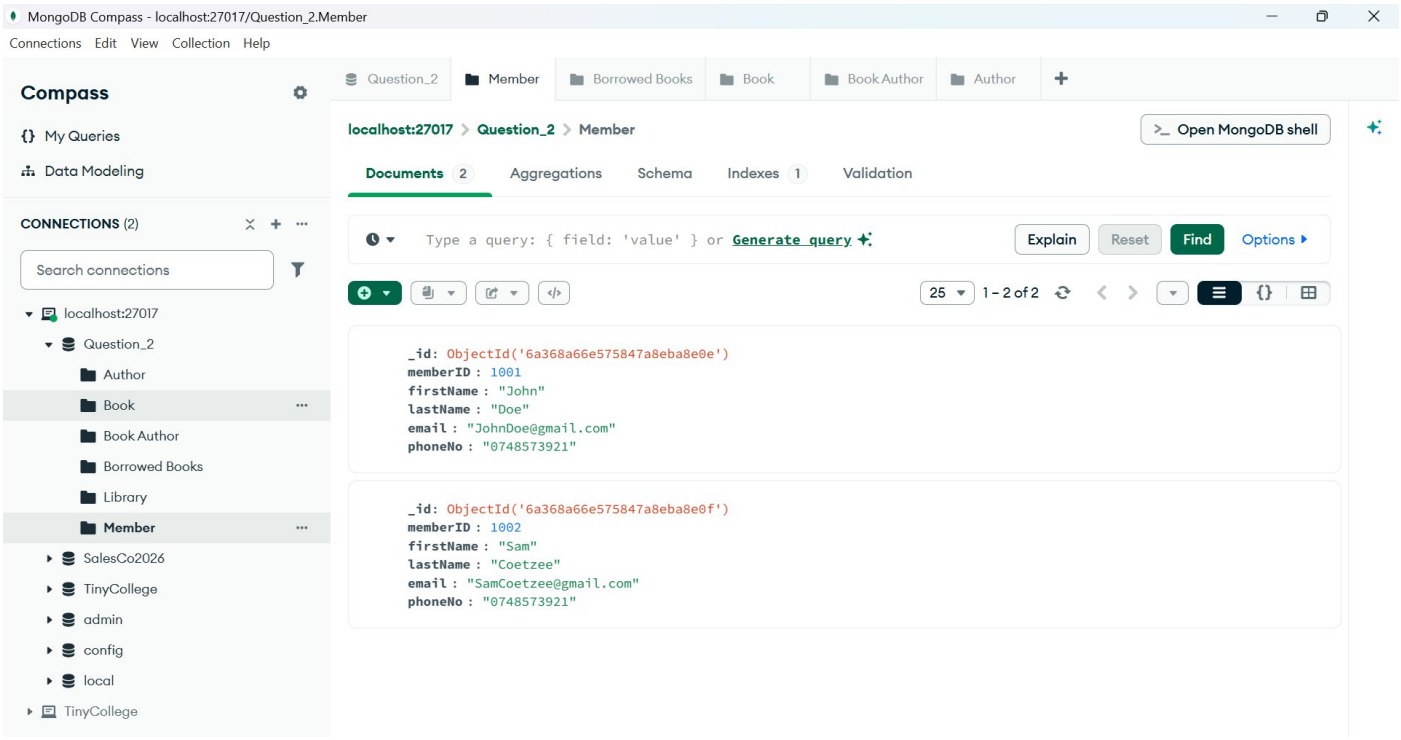
2.1)



2.2)

```
/**
* Paste one or more documents here
*/
[
  {
    "memberID": 1001,
    "firstName": "John",
    "lastName": "Doe",
    "email": "JohnDoe@gmail.com",
    "phoneNo": "0748573921"
  },
  {
    "memberID": 1002,
    "firstName": "Sam",
    "lastName": "Coetzee",
    "email": "SamCoetzee@gmail.com",
    "phoneNo": "0748573921"
  }
```

]



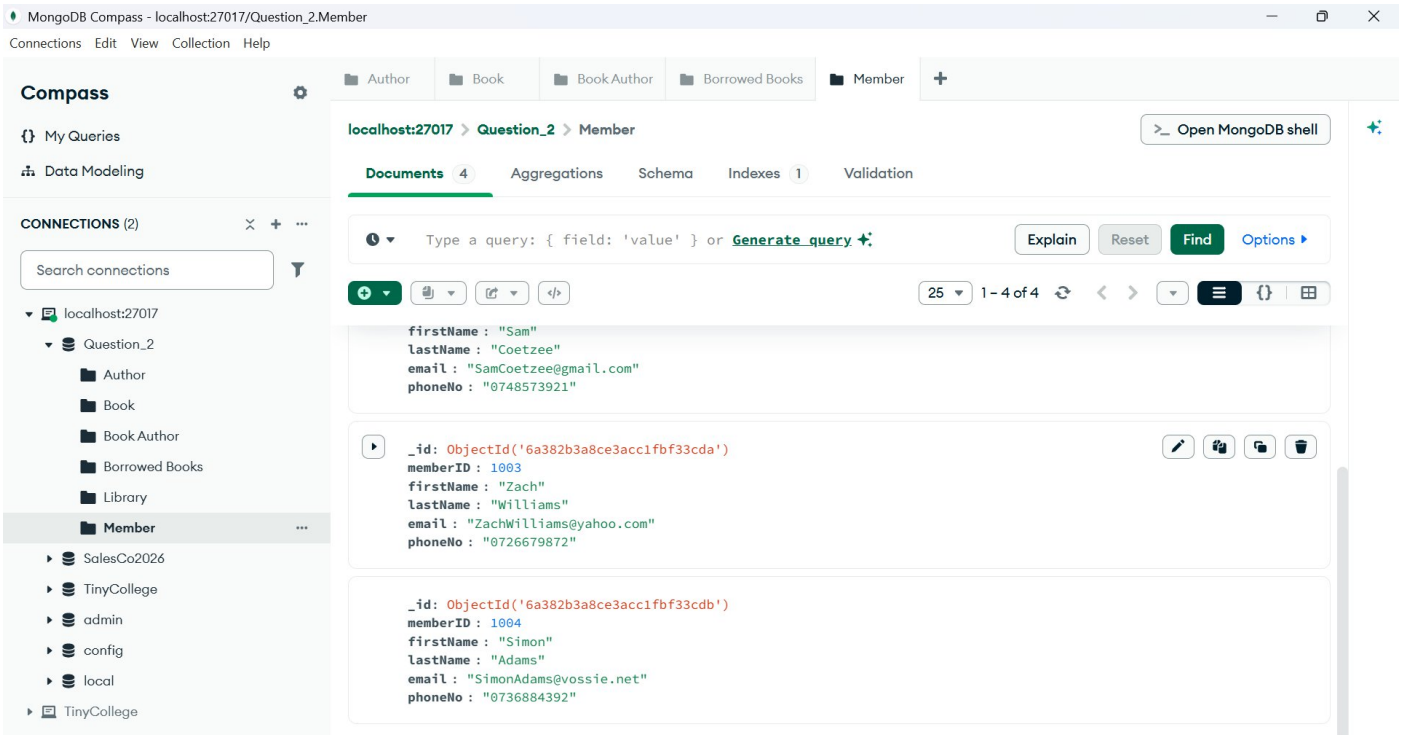
/**

* Paste one or more documents here

*/


```
[
  {
    "memberID": 1003,
    "firstName": "Zach",
    "lastName": "Williams",
    "email": "ZachWilliams@yahoo.com",
    "phoneNo": "0726679872"
  },
  {
    "memberID": 1004,
    "firstName": "Simon",
    "lastName": "Adams",
    "email": "SimonAdams@vossie.net",
    "phoneNo": "0736884392"
  }
```

]



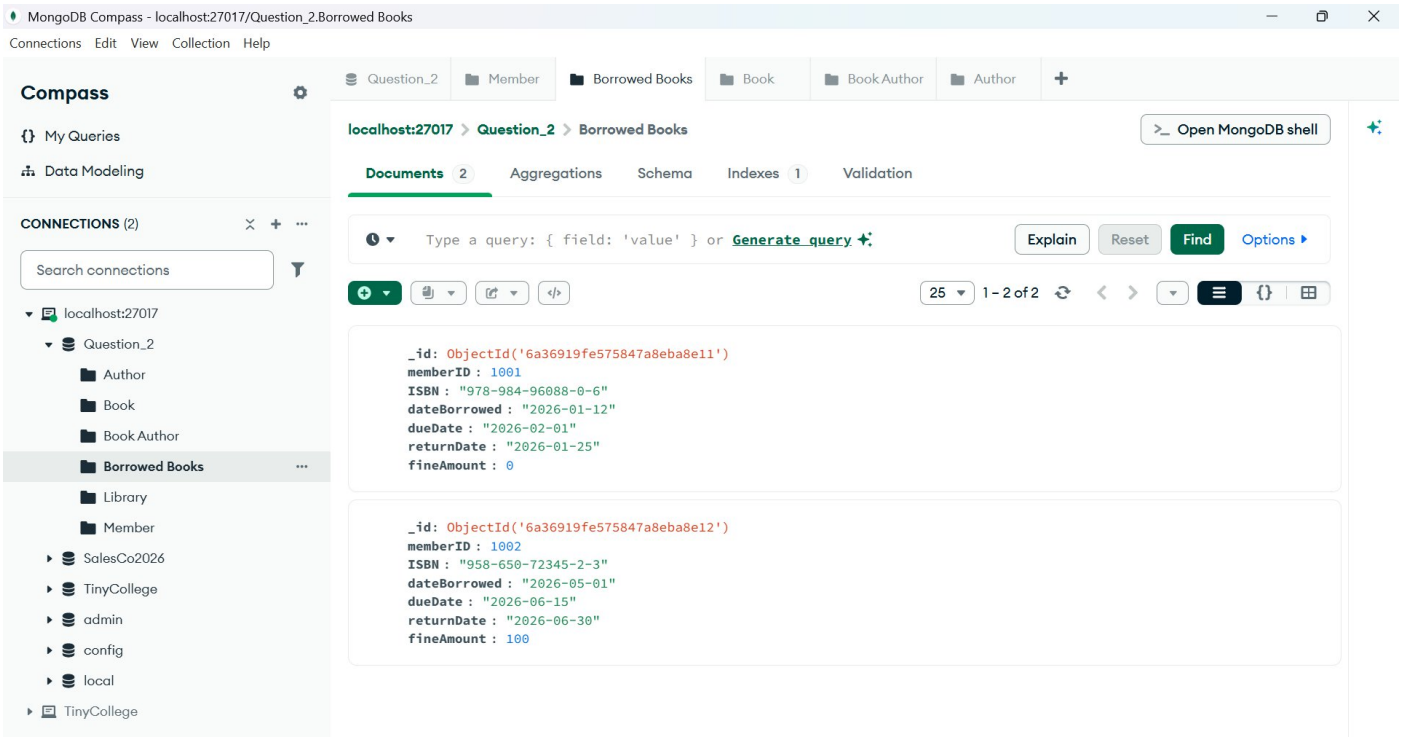
/**

* Paste one or more documents here

*/


```
[
  {
    "memberID": 1001,
    "ISBN": "978-984-96088-0-6",
    "dateBorrowed": "2026-01-12",
    "dueDate": "2026-02-01",
    "returnDate": "2026-01-25",
    "fineAmount": 0
  },
  {
    "memberID": 1002,
    "ISBN": "958-650-72345-2-3",
    "dateBorrowed": "2026-05-01",
    "dueDate": "2026-06-15",
    "returnDate": "2026-06-30",
    "fineAmount": 100
  }
```

]



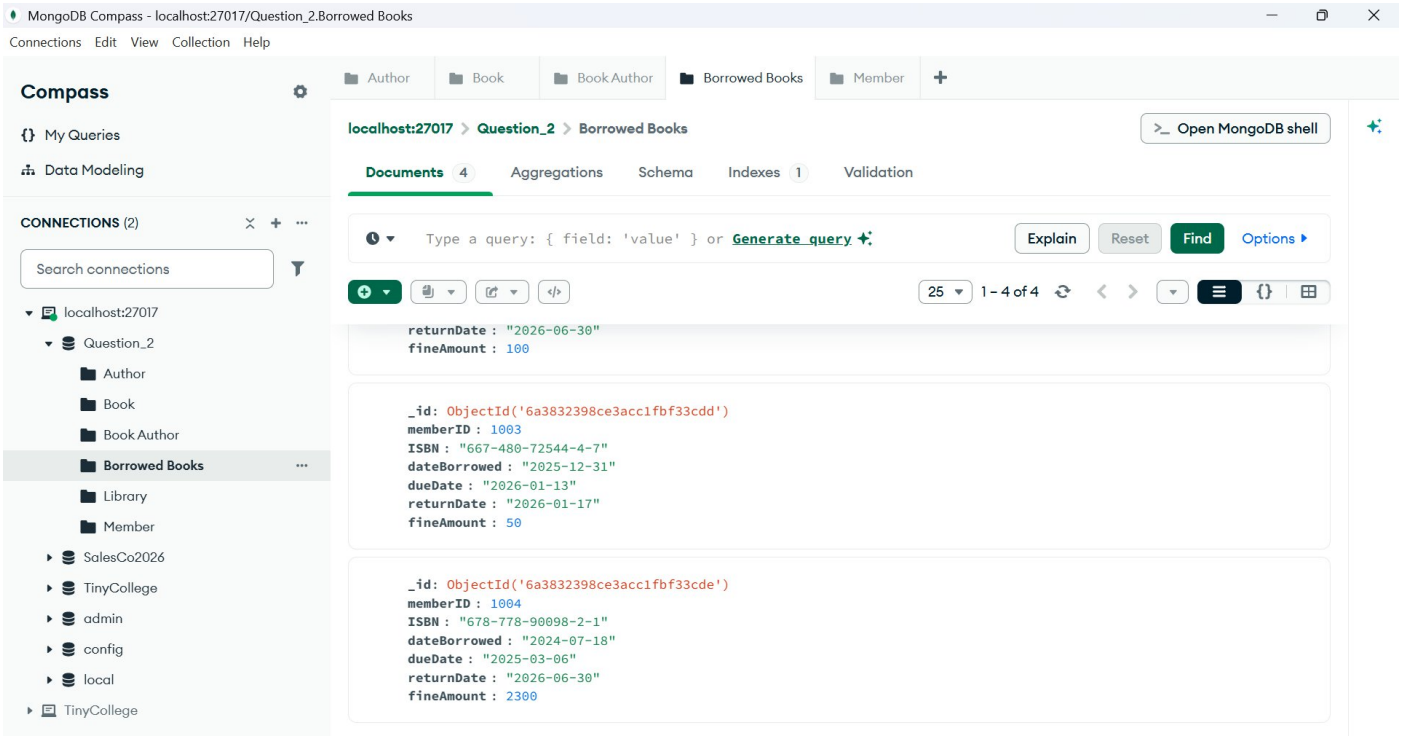
/**

* Paste one or more documents here

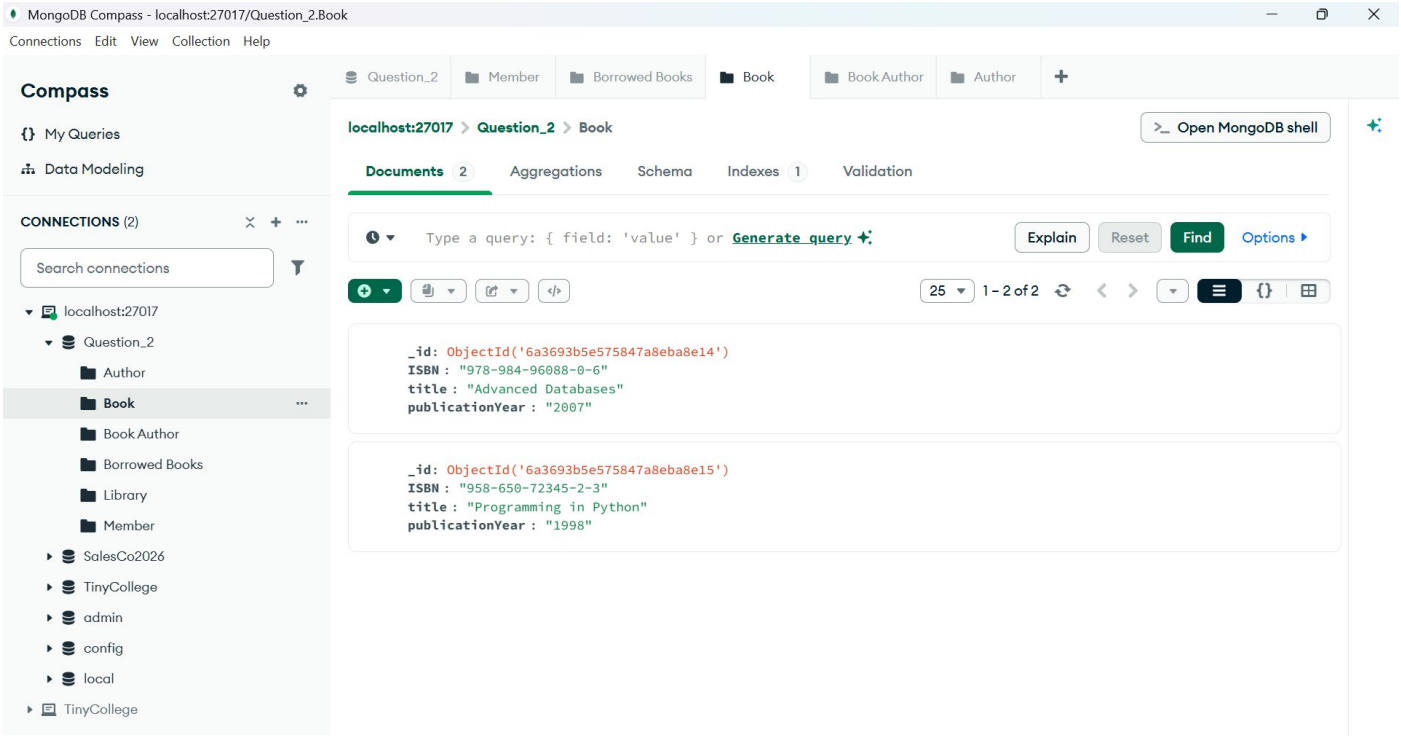
*/


```
[
  {
    "memberID": 1003,
    "ISBN": "667-480-72544-4-7",
    "dateBorrowed": "2025-12-31",
    "dueDate": "2026-01-13",
    "returnDate": "2026-01-17",
    "fineAmount": 50
  },
  {
    "memberID": 1004,
    "ISBN": "678-778-90098-2-1",
    "dateBorrowed": "2024-07-18",
    "dueDate": "2025-03-06",
    "returnDate": "2026-06-30",
    "fineAmount": 2300
```

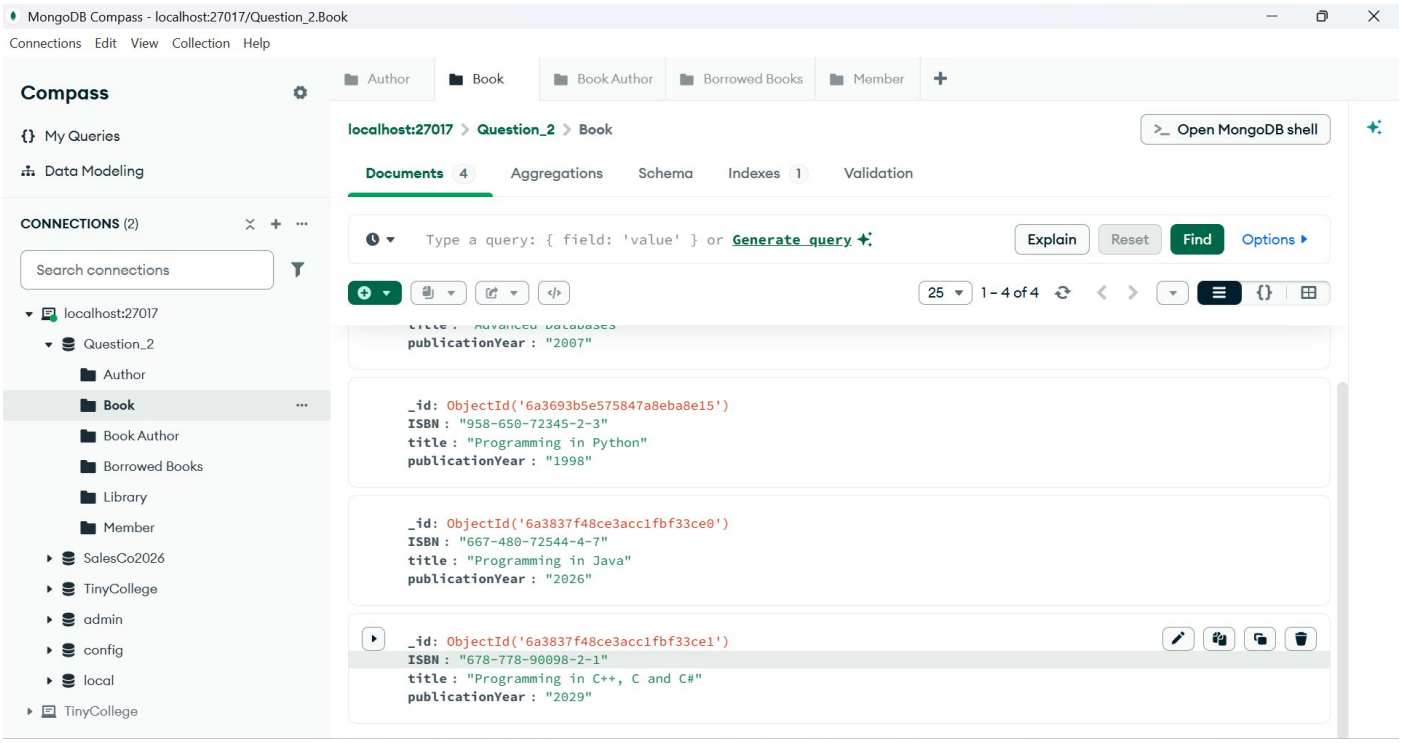
```
  }
]
```



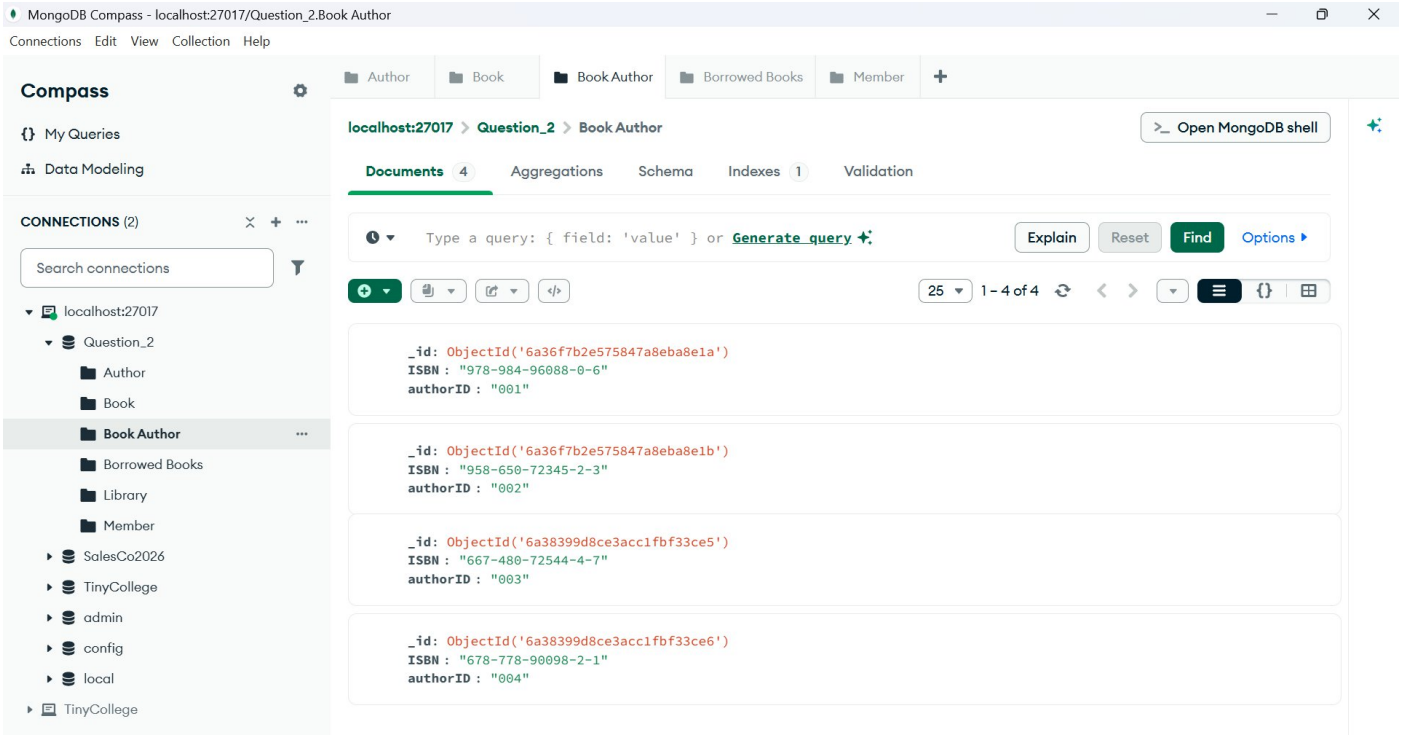

```
/**
* Paste one or more documents here
*/
[
  {
    "ISBN": "978-984-96088-0-6",
    "title": "Advanced Databases",
    "publicationYear": "2007"
  },
  {
    "ISBN": "958-650-72345-2-3",
    "title": "Programming in Python",
    "publicationYear": "1998",
  }
]
```



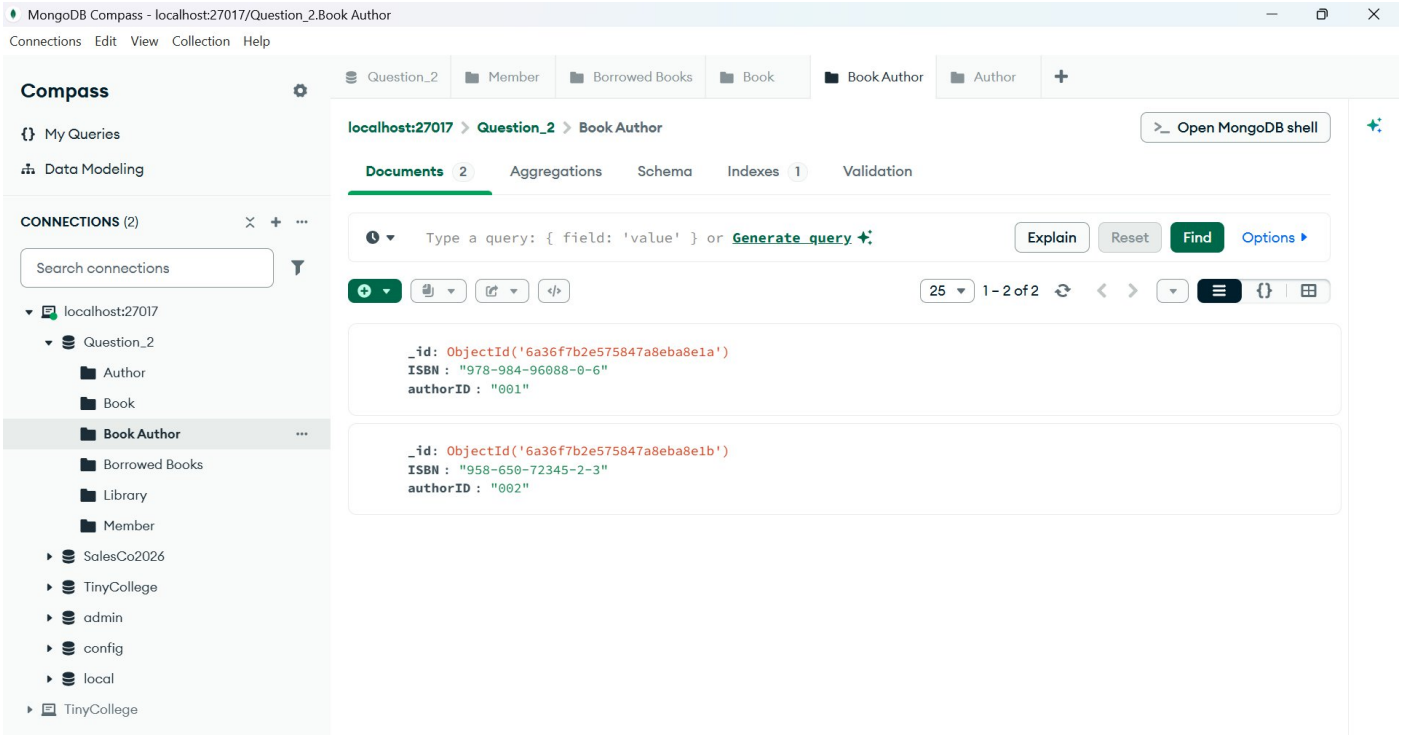

```
/**
* Paste one or more documents here
*/
[
  {
    "ISBN": "667-480-72544-4-7",
    "title": "Programming in Java",
    "publicationYear": "2026"
  },
  {
    "ISBN": "678-778-90098-2-1",
    "title": "Programming in C++, C and C#",
    "publicationYear": "2029",
  }
]
```



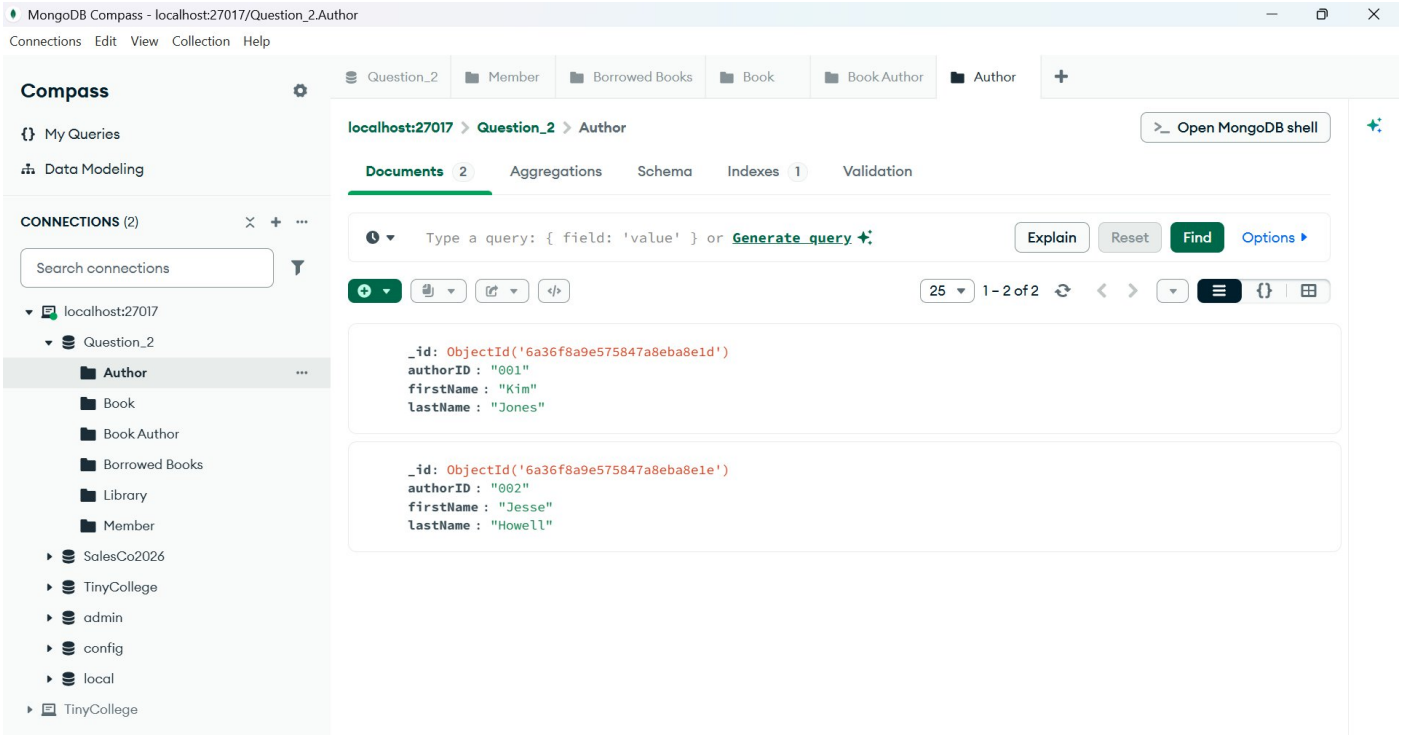

```
/**
* Paste one or more documents here
*/
[
  {
    "ISBN": "667-480-72544-4-7",
    "authorID": "003"
  },
  {
    "ISBN": "678-778-90098-2-1",
    "authorID": "004"
  }
]
```



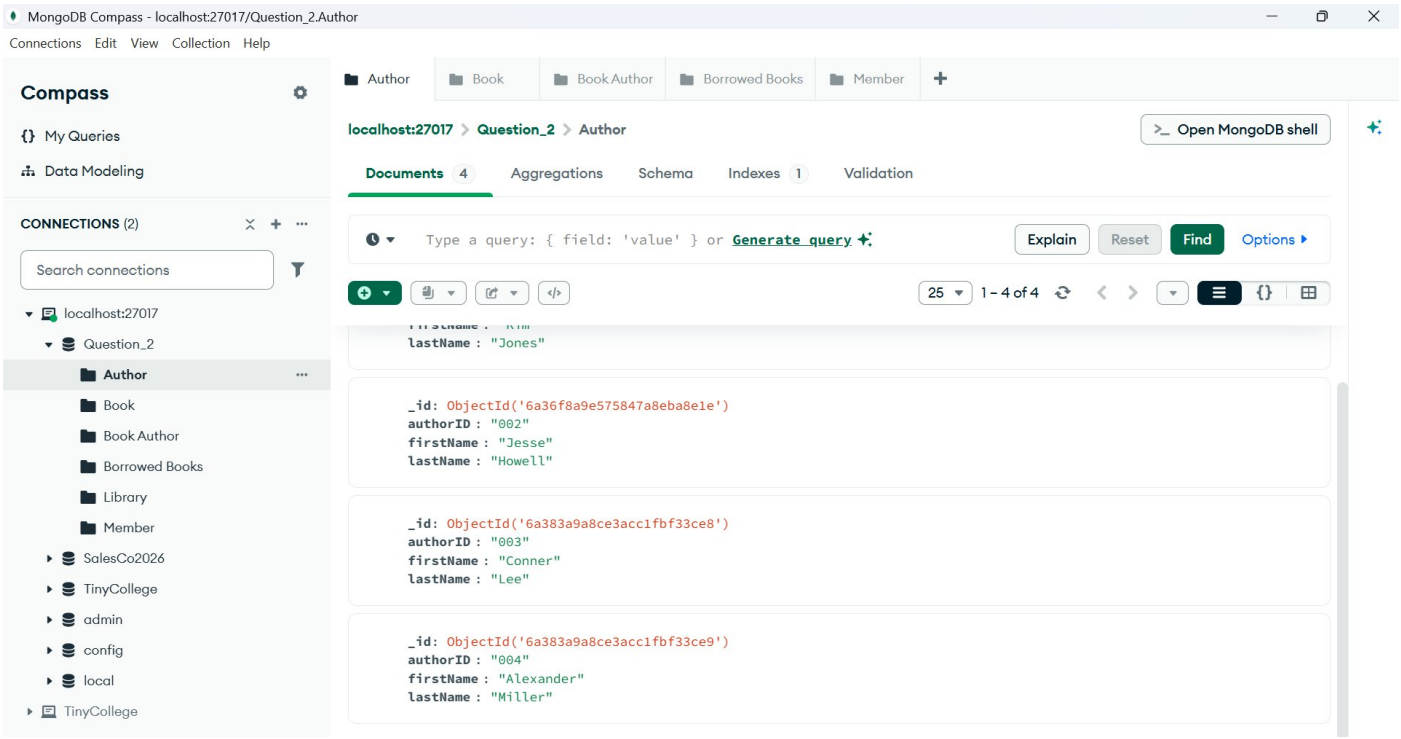

```
/**
* Paste one or more documents here
*/
[
  {
    "ISBN": "978-984-96088-0-6",
    "authorID": "001"
  },
  {
    "ISBN": "958-650-72345-2-3",
    "authorID": "002"
  }
]
```




```
/**
* Paste one or more documents here
*/
[
  {
    "authorID": "001",
    "firstName": "Kim",
    "lastName": "Jones"
  },
  {
    "authorID": "002",
    "firstName": "Jesse",
    "lastName": "Howell"
  }
]
```

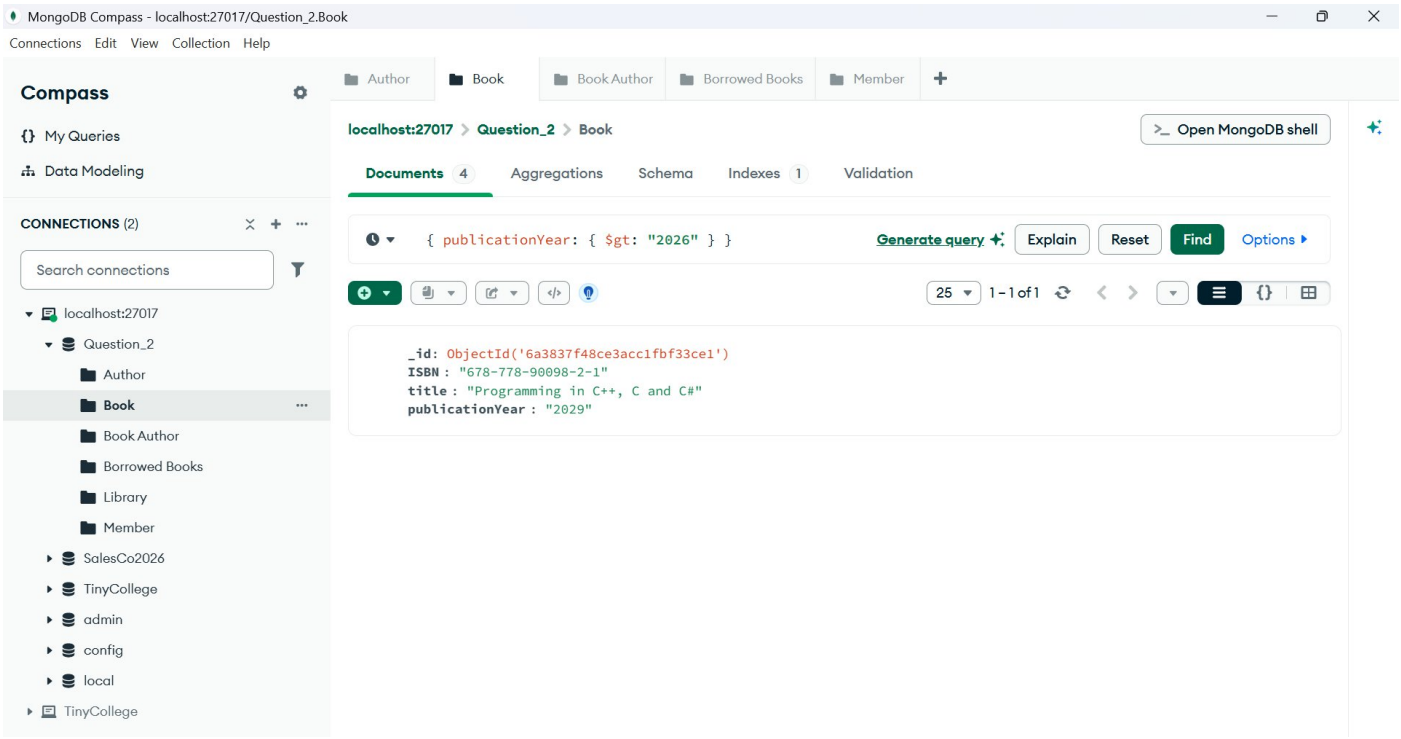



```
/**
* Paste one or more documents here
*/
[
  {
    "authorID": "003",
    "firstName": "Conner",
    "lastName": "Lee"
  },
  {
    "authorID": "004",
    "firstName": "Alexander",
    "lastName": "Miller"
  }
]
```



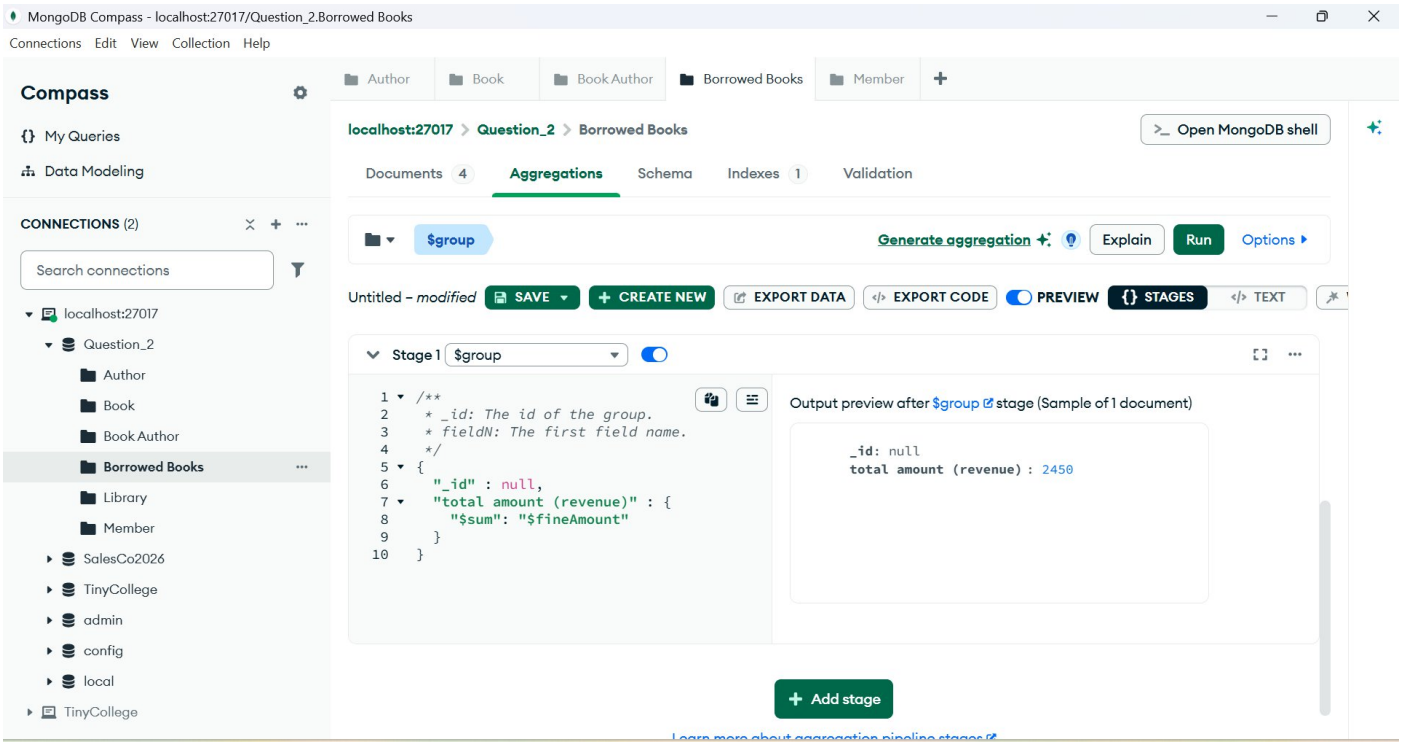
2.3)

```
{
  filter: {
    publicationYear: {
      $gt: '2026'
    }
  }
}
```

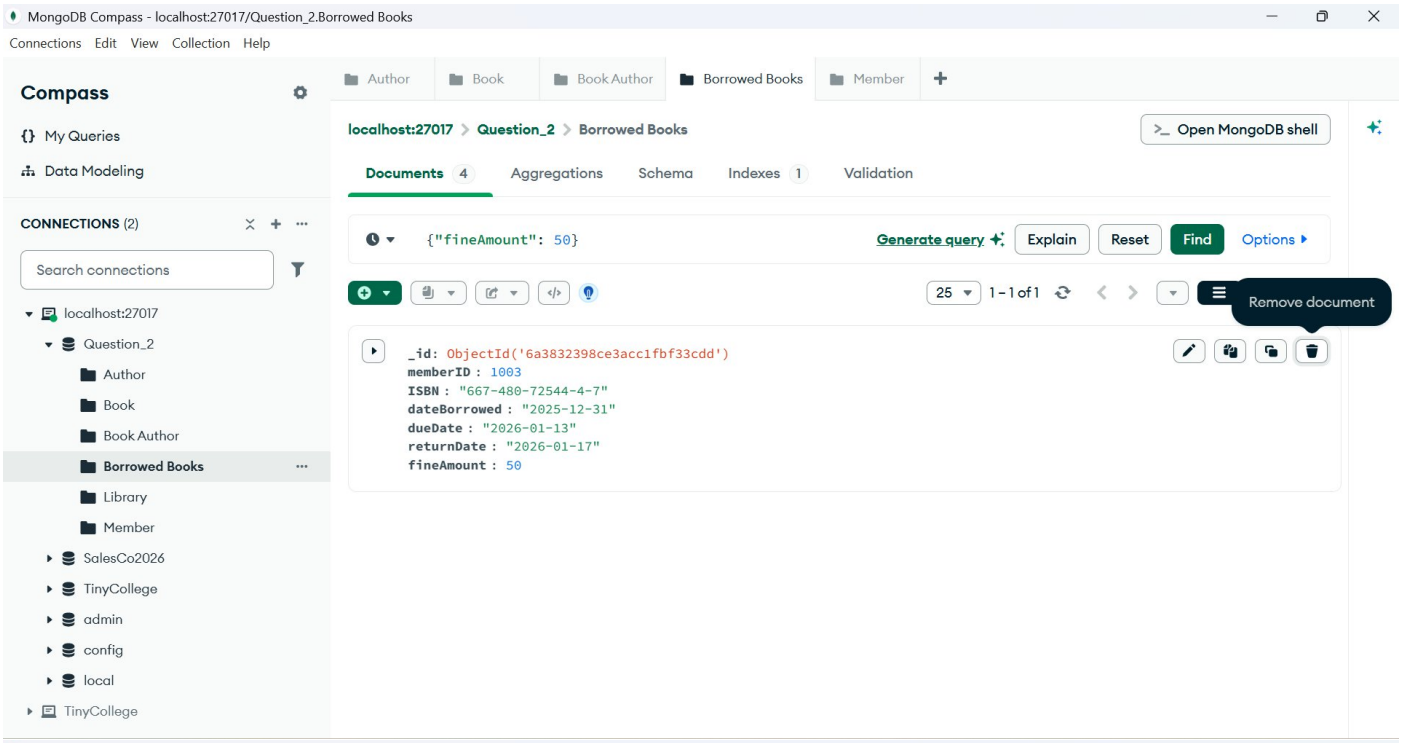


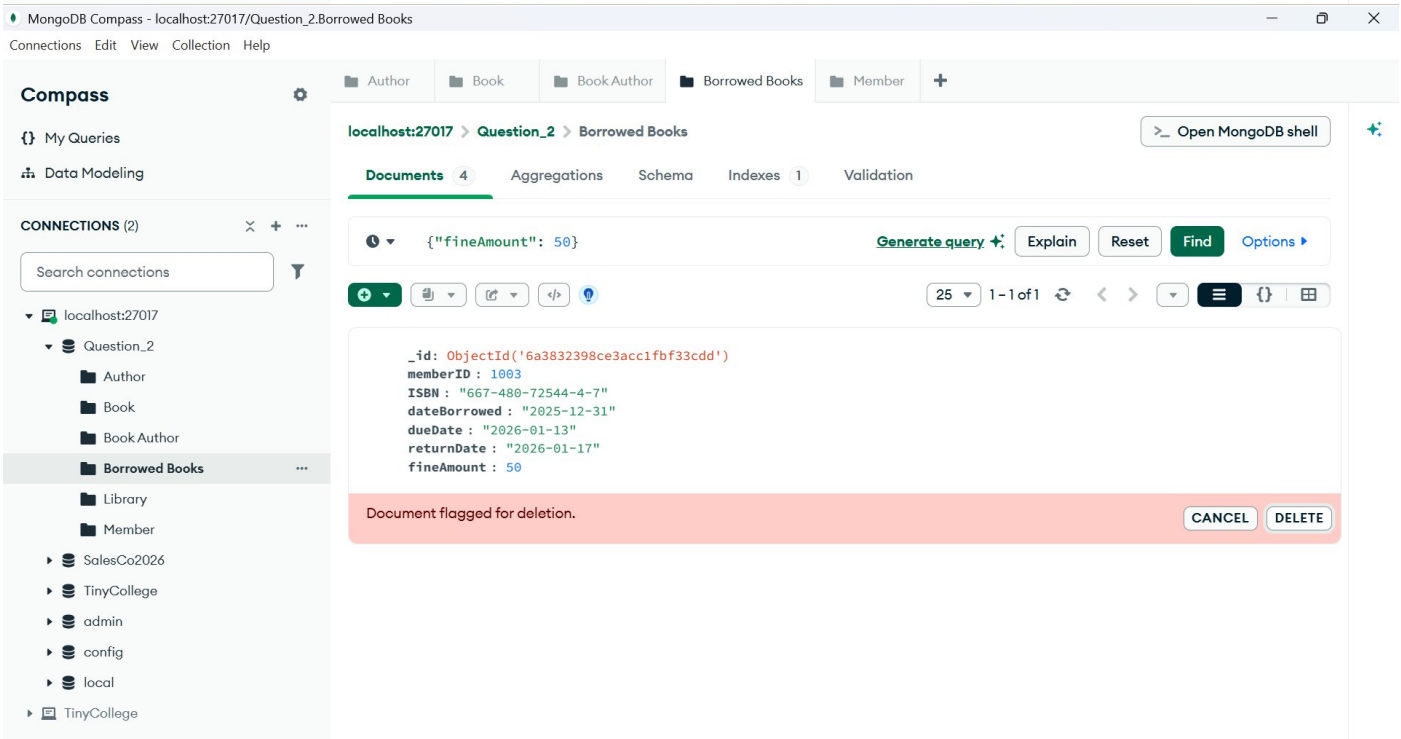
2.4)

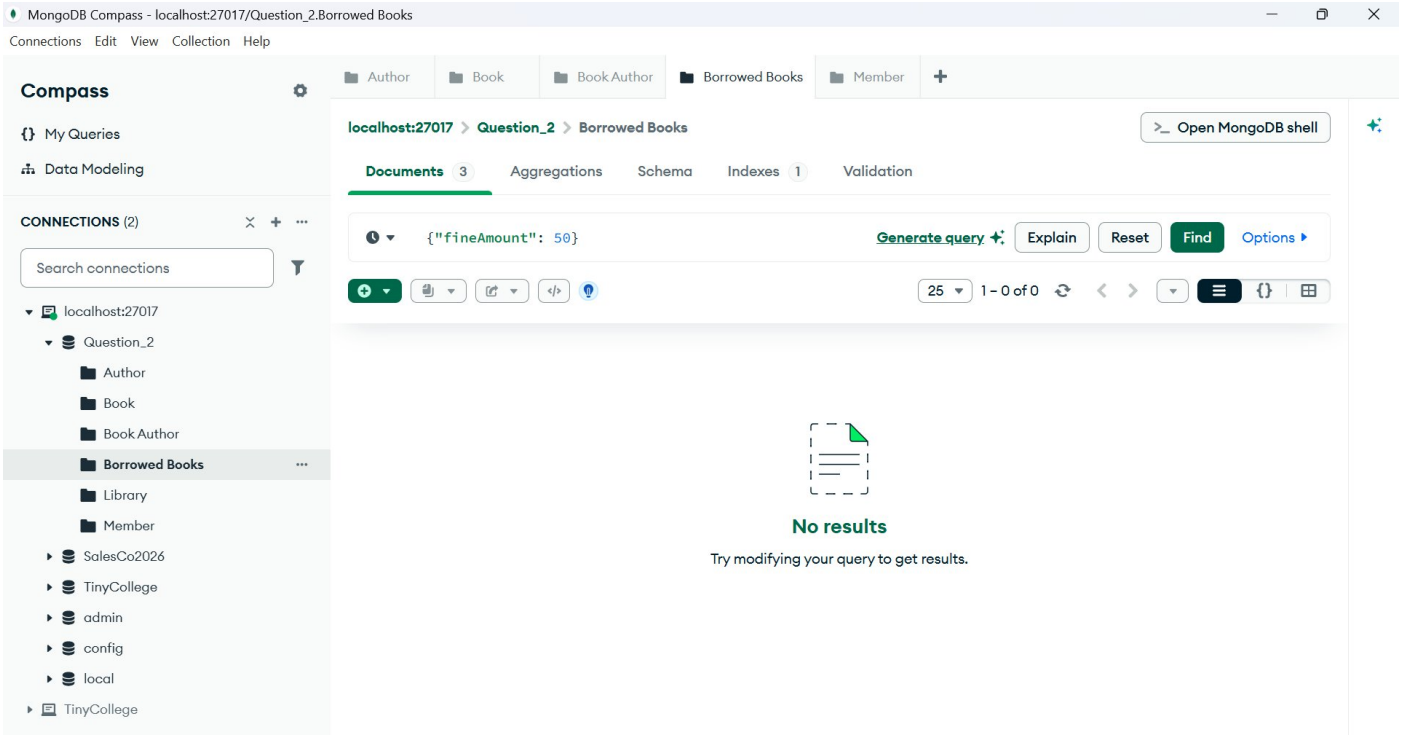
```
/**
 * _id: The id of the group.
 * fieldN: The first field name.
 */
{
  “_id” : null,
  "total amount (revenue)" : {
    "$sum": "$fineAmount"
  }
}
```



2.5)

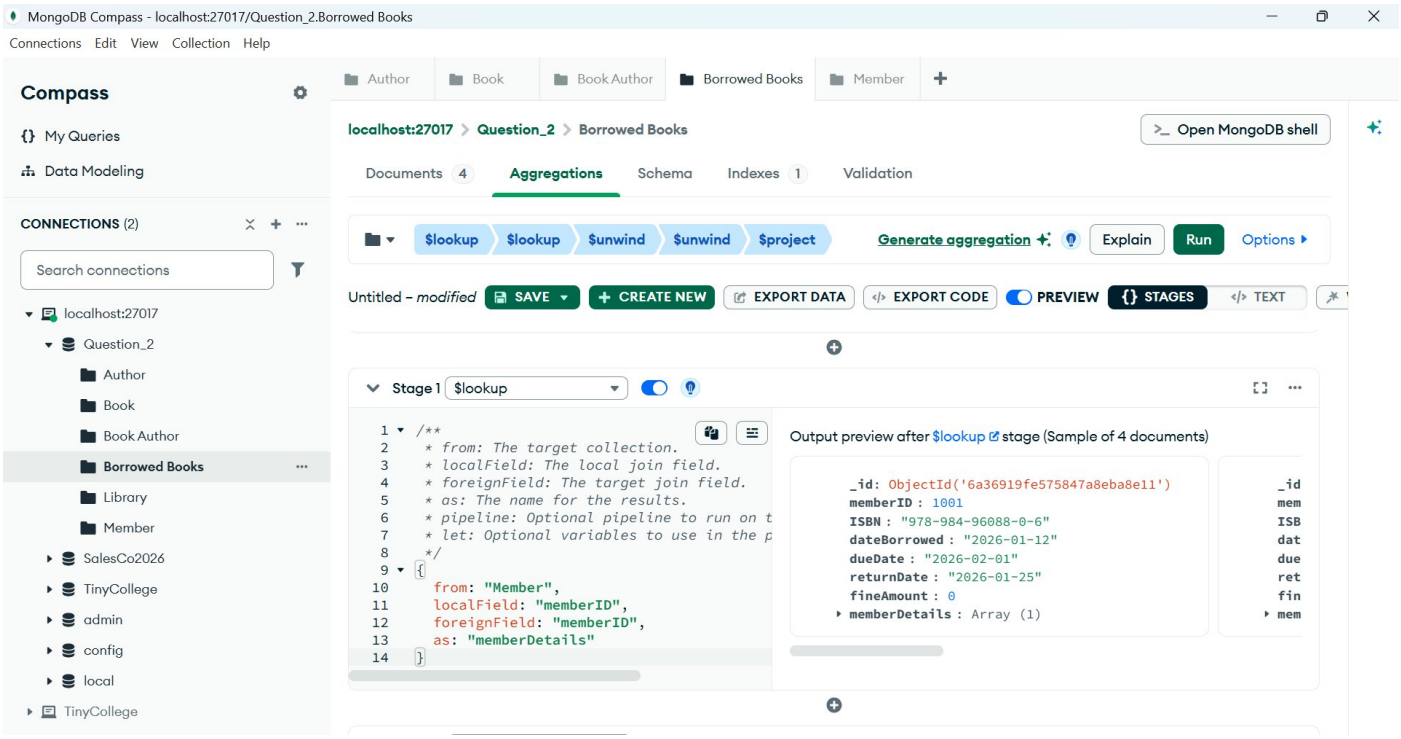

MongoDB Compass - localhost:27017/Question_2.Borrowed Books
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
Question_2
Author
Book
Book Author
Borrowed Books
Library
Member
SalesCo2026
TinyCollege
admin
config
local
TinyCollege
Author
Book
Book Author
Borrowed Books
Member
localhost:27017 > Question_2 > Borrowed Books
Open MongoDB shell
Documents 4
Aggregations
Schema
Indexes 1
Validation
{"fineAmount": 50}
Generate query
Explain
Reset
Find
Options
25
1 - 1 of 1
Remove document
_id: ObjectId('6a3832398ce3acc1fbf33cdd')
memberID : 1003
ISBN : "667-480-72544-4-7"
dateBorrowed : "2025-12-31"
dueDate : "2026-01-13"
returnDate : "2026-01-17"
fineAmount : 50


MongoDB Compass - localhost:27017/Question_2.Borrowed Books
Connections Edit View Collection Help
Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
localhost:27017
Question_2
Author
Book
Book Author
Borrowed Books
Library
Member
SalesCo2026
TinyCollege
admin
config
local
TinyCollege
Author
Book
Book Author
Borrowed Books
Member
localhost:27017 > Question_2 > Borrowed Books
Open MongoDB shell
Documents 4
Aggregations
Schema
Indexes 1
Validation
{"fineAmount": 50}
Generate query
Explain
Reset
Find
Options
25
1 - 1 of 1
_id: ObjectId('6a3832398ce3acc1fbf33cdd')
memberID : 1003
ISBN : "667-480-72544-4-7"
dateBorrowed : "2025-12-31"
dueDate : "2026-01-13"
returnDate : "2026-01-17"
fineAmount : 50
Document flagged for deletion.
CANCEL
DELETE

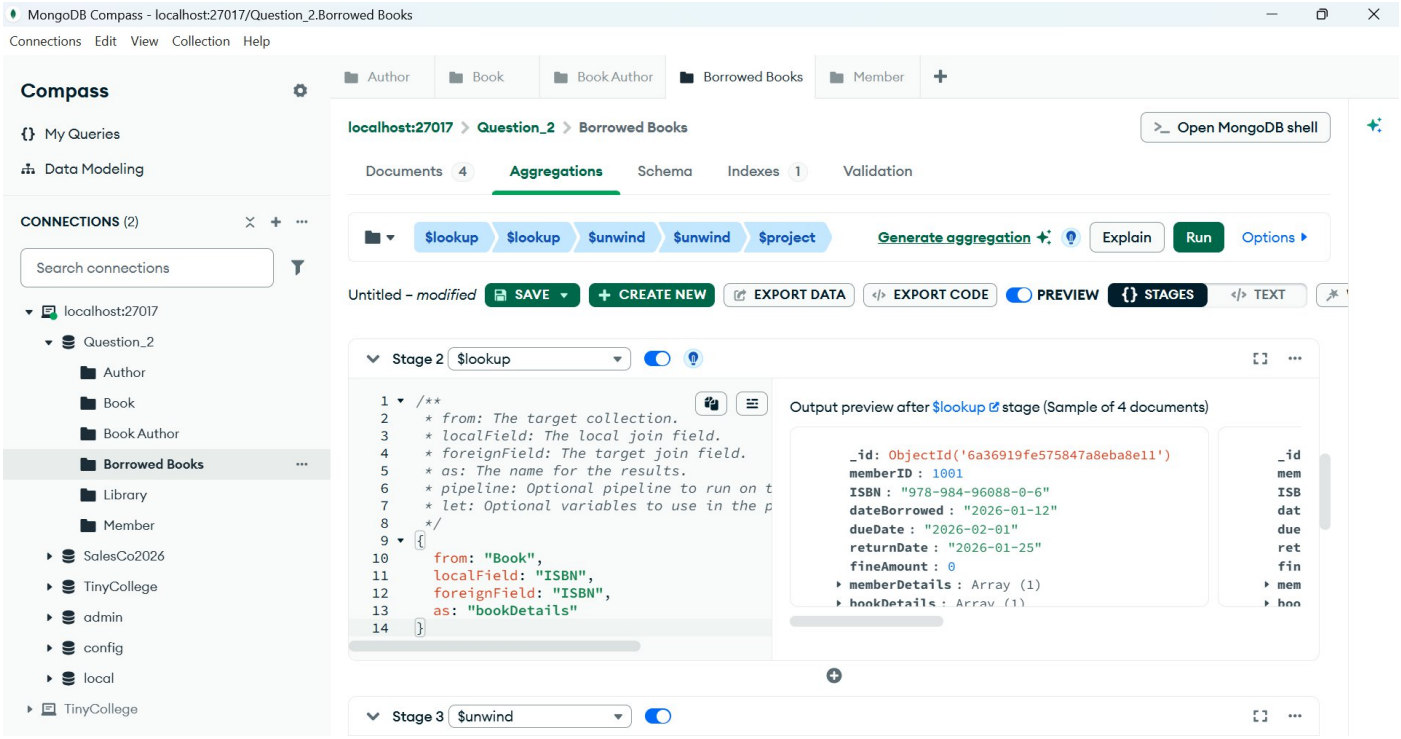


2.6)

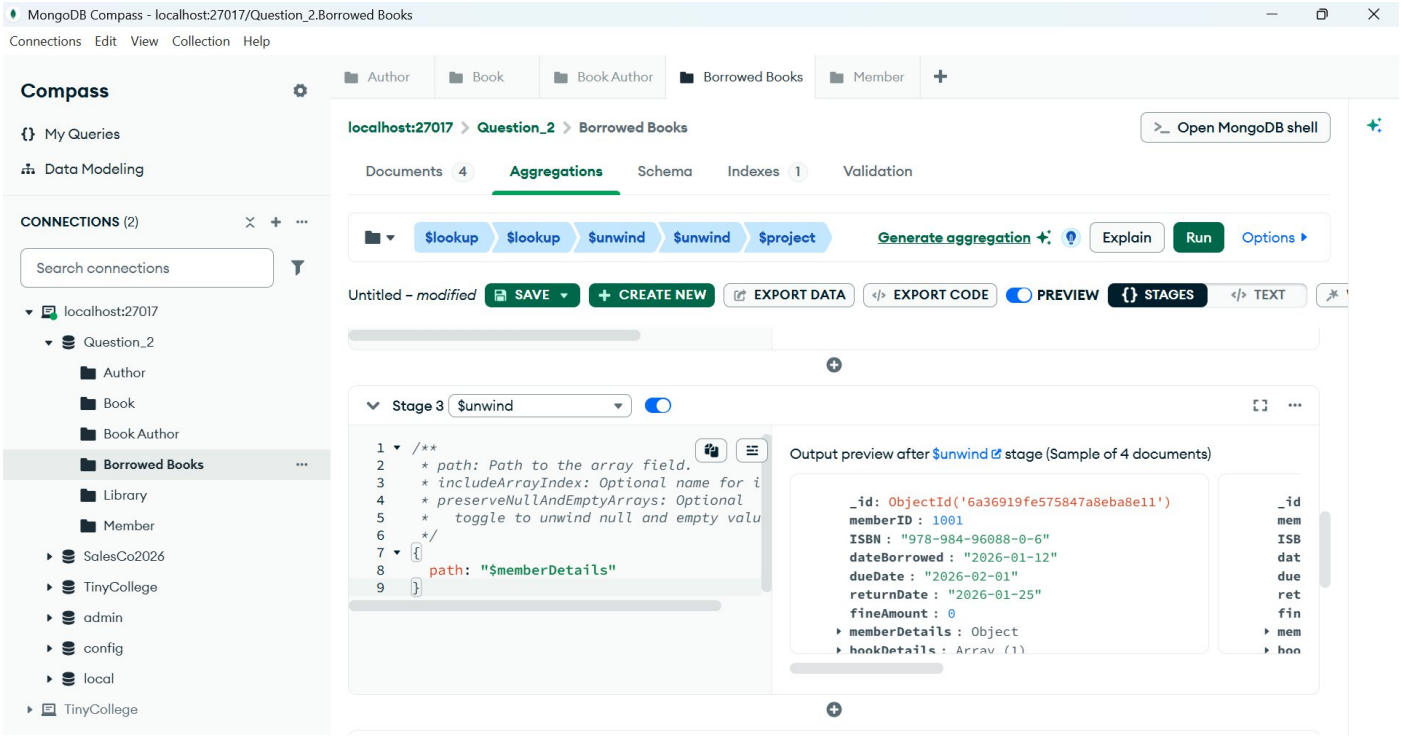
```
/**
 * from: The target collection.
 * localField: The local join field.
 * foreignField: The target join field.
 * as: The name for the results.
 * pipeline: Optional pipeline to run on the foreign collection.
 * let: Optional variables to use in the pipeline field stages.
 */
{
  from: "Member",
  localField: "memberID",
  foreignField: "memberID",
  as: "memberDetails"
}
```



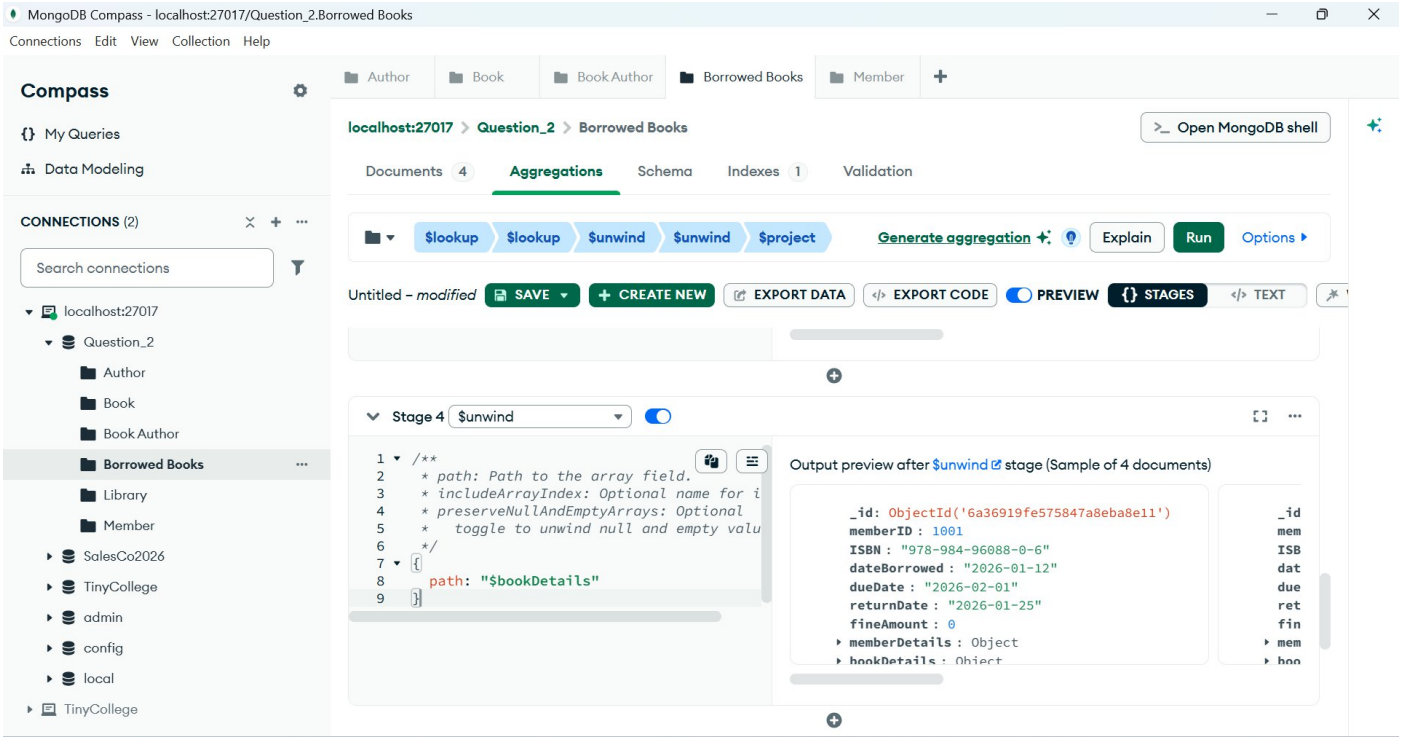

```
/**
 * from: The target collection.
 * localField: The local join field.
 * foreignField: The target join field.
 * as: The name for the results.
 * pipeline: Optional pipeline to run on the foreign collection.
 * let: Optional variables to use in the pipeline field stages.
 */
{
  from: "Book",
  localField: "ISBN",
  foreignField: "ISBN",
  as: "bookDetails"
}
```



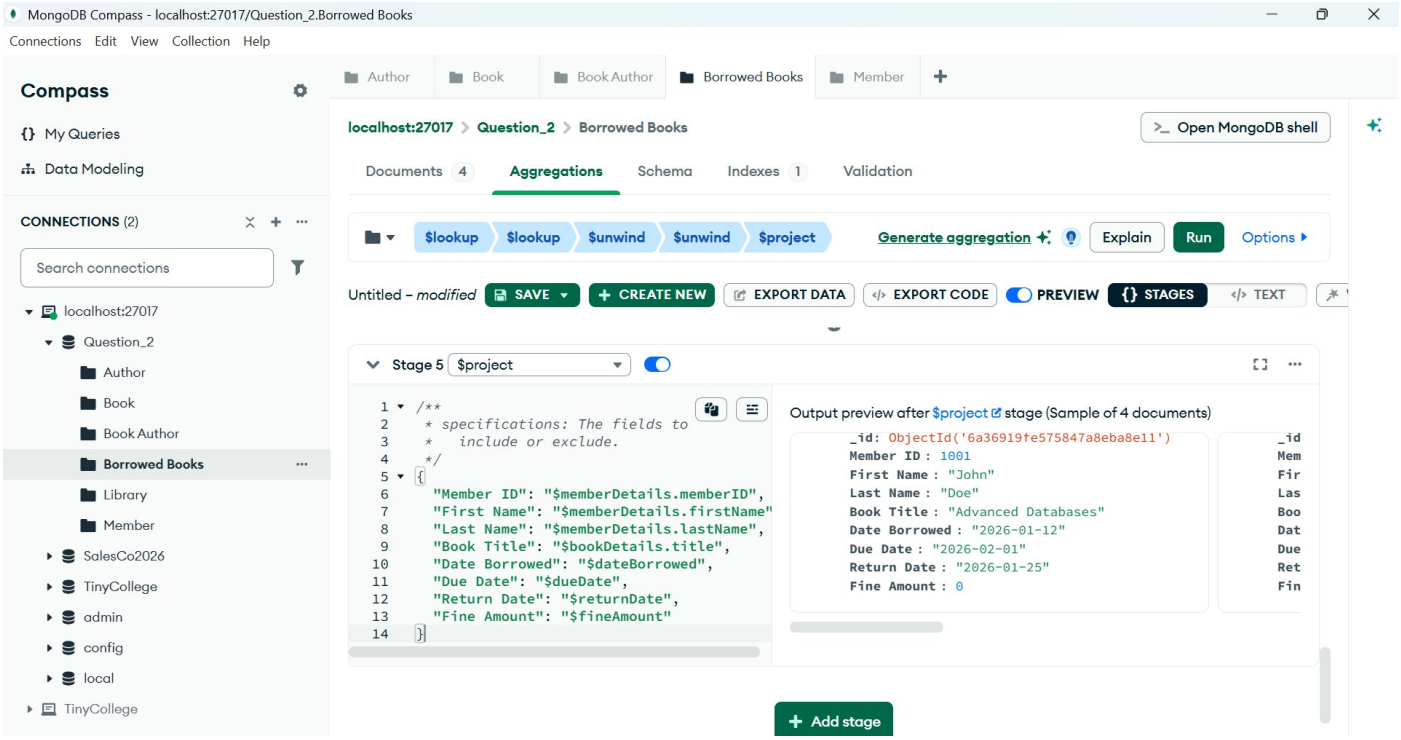

```
/**
 * path: Path to the array field.
 * includeArrayIndex: Optional name for index.
 * preserveNullAndEmptyArrays: Optional
 *   toggle to unwind null and empty values.
 */
{
  path: "$memberDetails"
}
```




```
/**
 * path: Path to the array field.
 * includeArrayIndex: Optional name for index.
 * preserveNullAndEmptyArrays: Optional
 *   toggle to unwind null and empty values.
 */
{
  path: "$bookDetails"
}
```



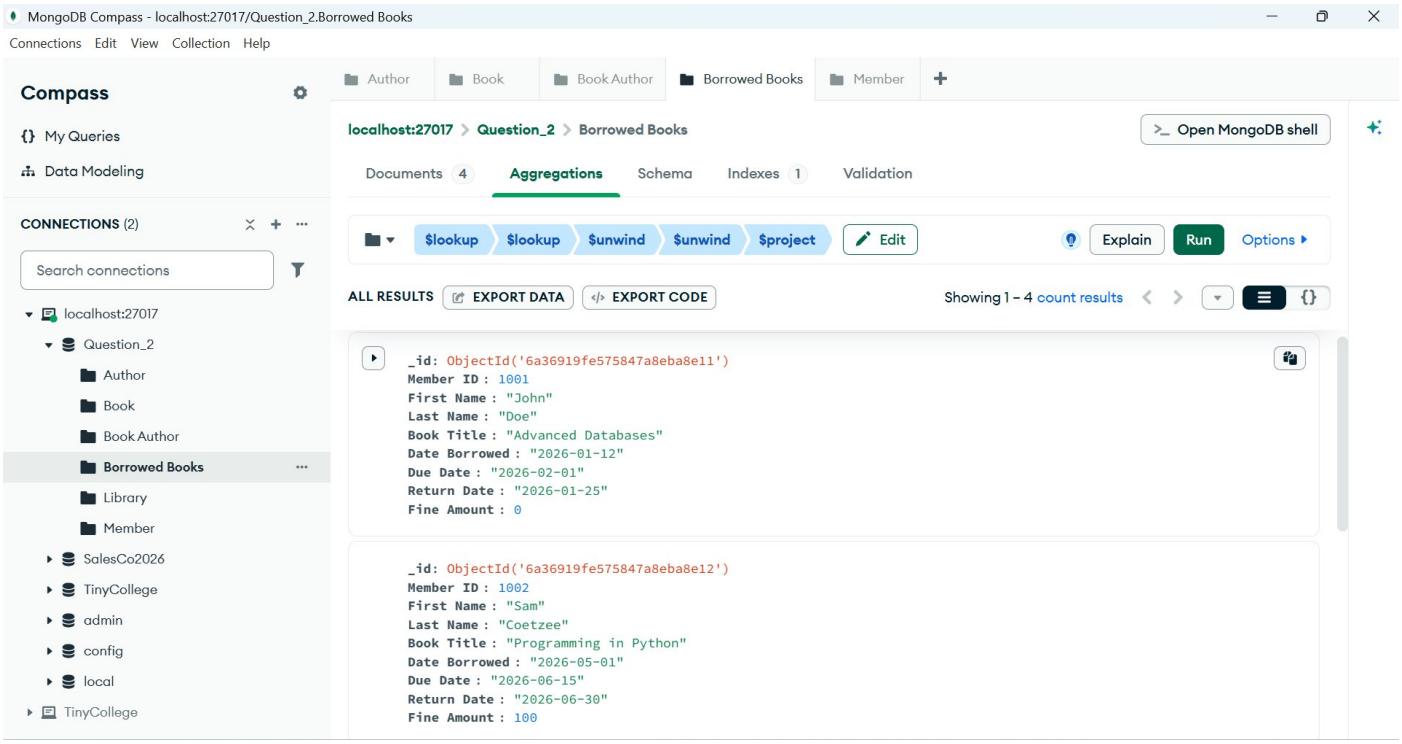

```
/**
 * specifications: The fields to
 *   include or exclude.
 */
{
  "Member ID": "$memberDetails.memberID",
  "First Name": "$memberDetails.firstName",
  "Last Name": "$memberDetails.lastName",
  "Book Title": "$bookDetails.title",
  "Date Borrowed": "$dateBorrowed",
  "Due Date": "$dueDate",
  "Return Date": "$returnDate",
  "Fine Amount": "$fineAmount"
}
```



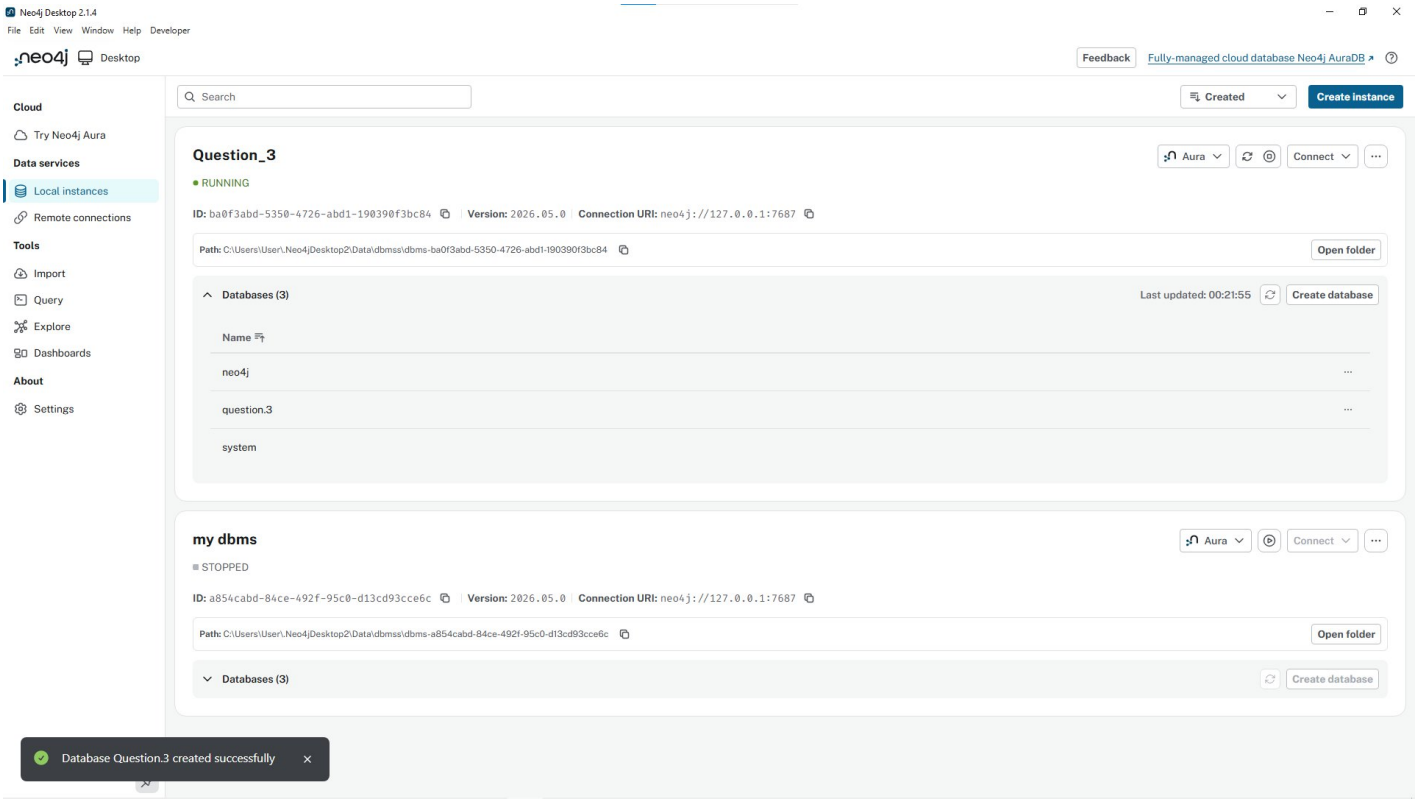
Node code:

```
[
  {
    '$lookup': {
      'from': 'Member',
      'localField': 'memberID',
      'foreignField': 'memberID',
      'as': 'memberDetails'
    }
  }, {
    '$lookup': {
      'from': 'Book',
      'localField': 'ISBN',
      'foreignField': 'ISBN',
      'as': 'bookDetails'
    }
  }, {
    '$unwind': {
      'path': '$memberDetails'
    }
```

```
  }, {
    '$unwind': {
      'path': '$bookDetails'
    }
  }, {
    '$project': {
      'Member ID': '$memberDetails.memberID',
      'First Name': '$memberDetails.firstName',
      'Last Name': '$memberDetails.lastName',
      'Book Title': '$bookDetails.title',
      'Date Borrowed': '$dateBorrowed',
      'Due Date': '$dueDate',
      'Return Date': '$returnDate',
      'Fine Amount': '$fineAmount'
    }
  }
]
```



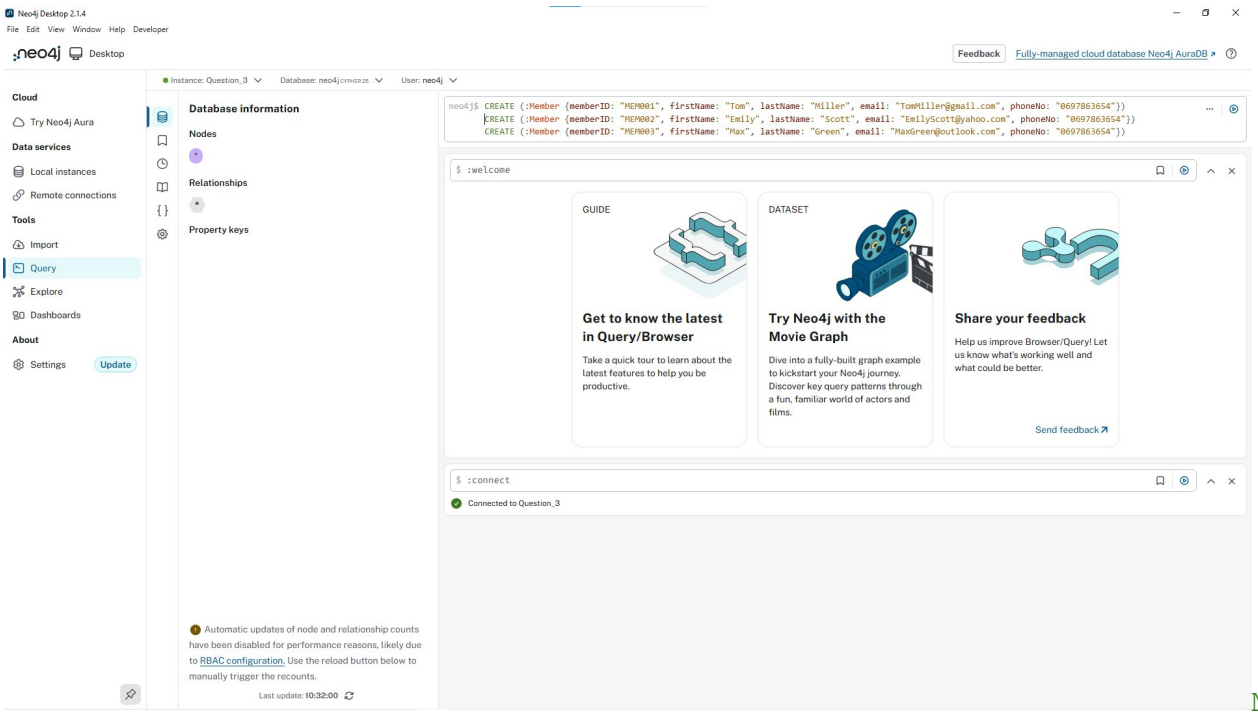
Deliverable 3 / Question 3:



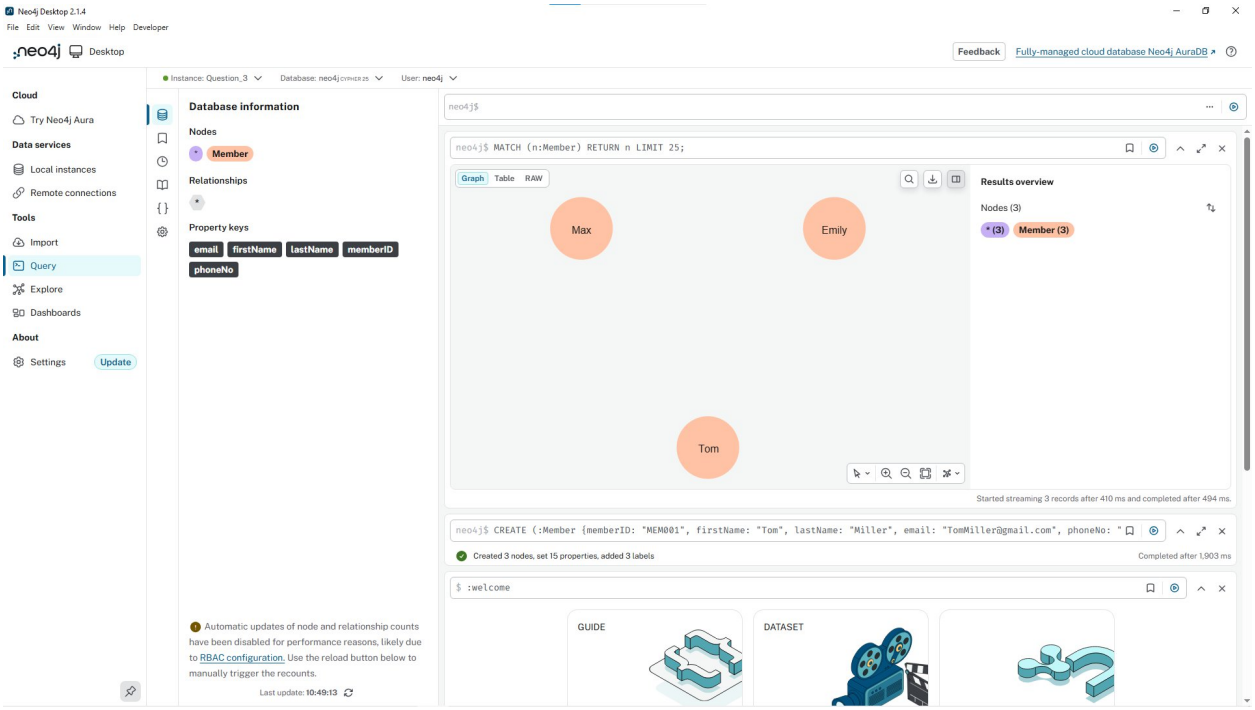
3.1.1)

(i)

```
CREATE (:Member {memberID: "MEM001", firstName: "Tom", lastName: "Miller", email: "TomMiller@gmail.com", phoneNo: "0697863654"}) CREATE (:Member {memberID: "MEM002", firstName: "Emily", lastName: "Scott", email: "EmilyScott@yahoo.com", phoneNo: "0697863654"}) CREATE (:Member {memberID: "MEM003", firstName: "Max", lastName: "Green", email: "MaxGreen@outlook.com", phoneNo: "0697863654"})
```




```
MATCH (n:Member) RETURN n;
```

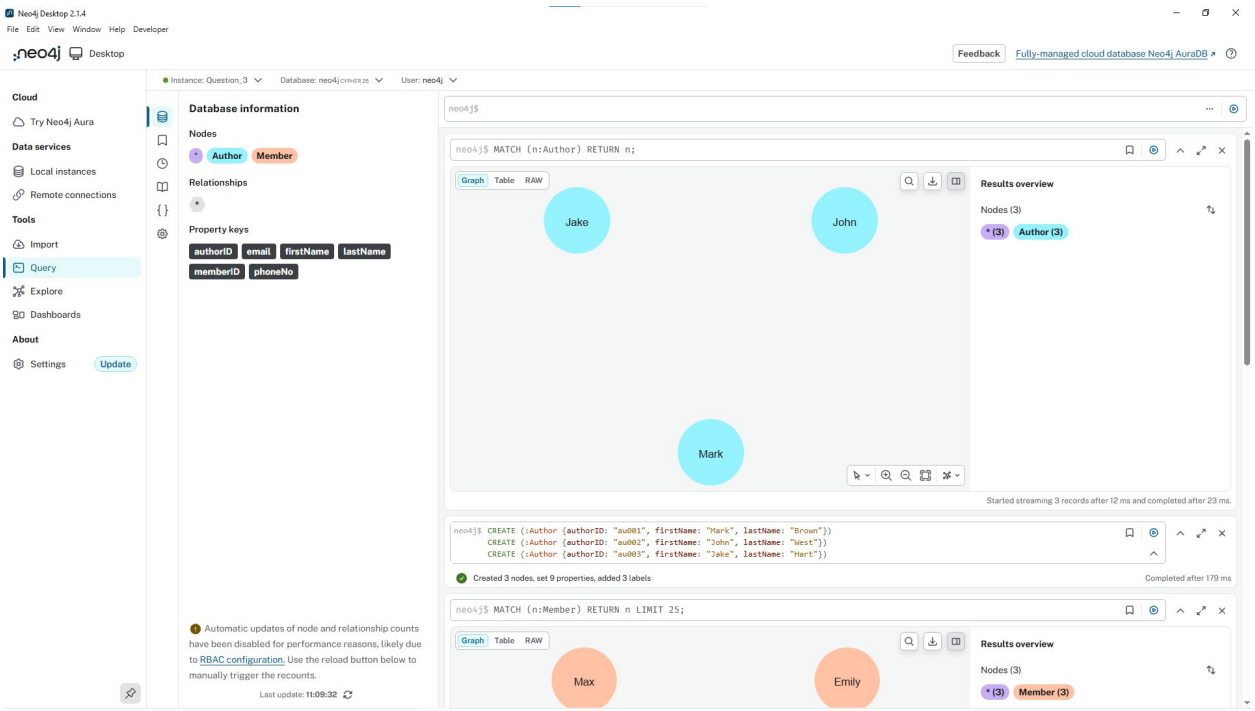


3.1.2)

(i)

```
CREATE (:Author {authorID: "au001", firstName: "Mark", lastName:
"Brown"}) CREATE (:Author {authorID: "au002", firstName: "John",
lastName: "West"}) CREATE (:Author {authorID: "au003", firstName:
"Jake", lastName: "Hart"})

MATCH (n:Author) RETURN n;
```



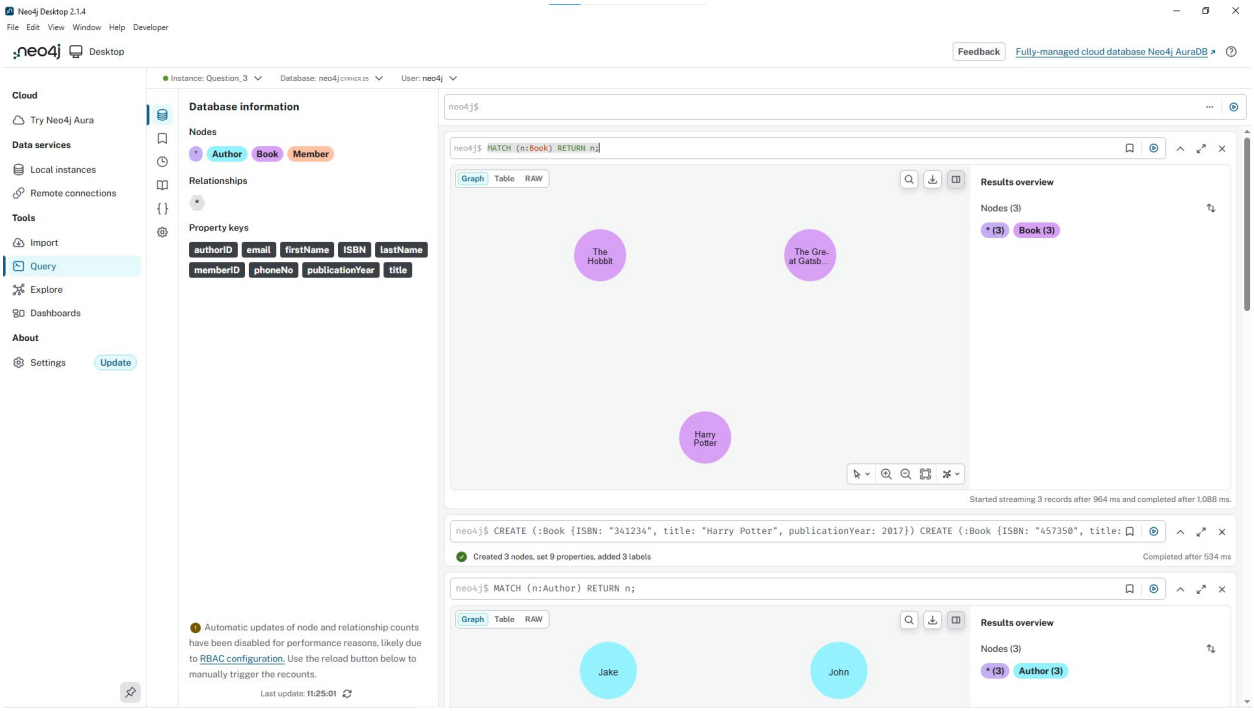
3.1.3)

(i)

```
CREATE (:Book {ISBN: "341234", title: "Harry Potter", publicationYear:
2017}) CREATE (:Book {ISBN: "457350", title: "The Great Gatsby",
```

```
publicationYear: 2000}) CREATE (:Book {ISBN: "678521", title: "The
Hobbit", publicationYear: 1997})


MATCH (n:Book) RETURN n;
```

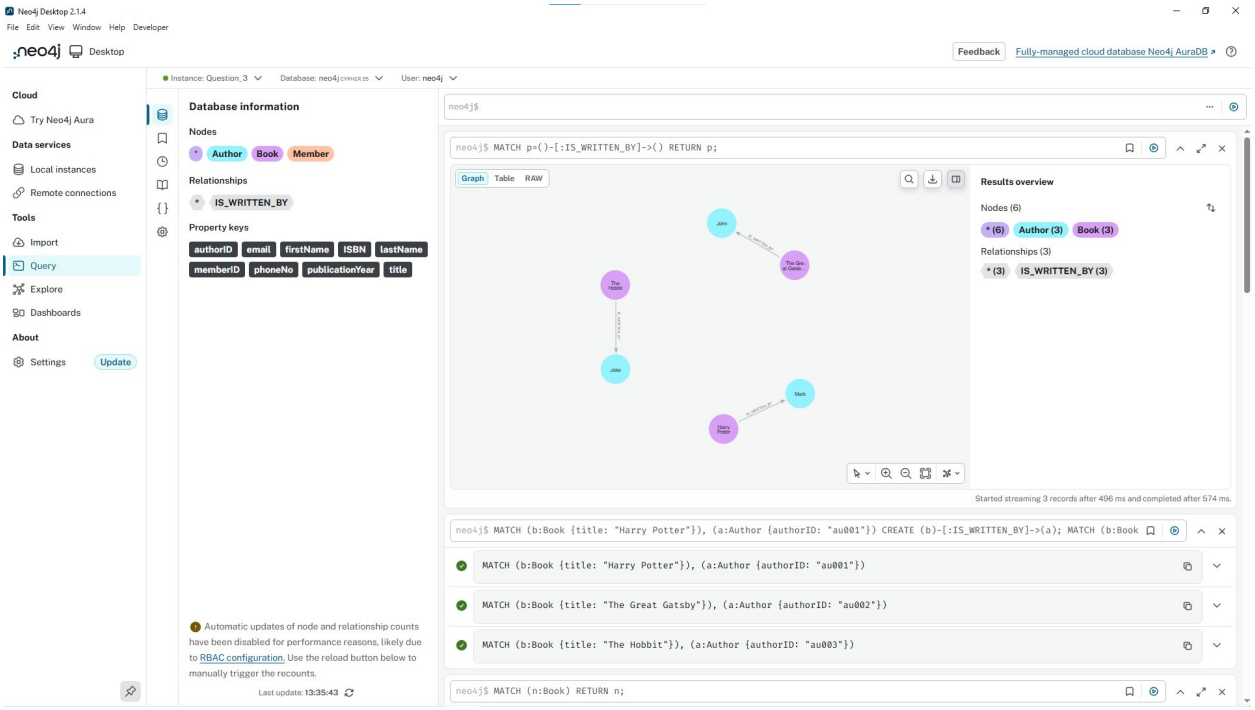


3.2)

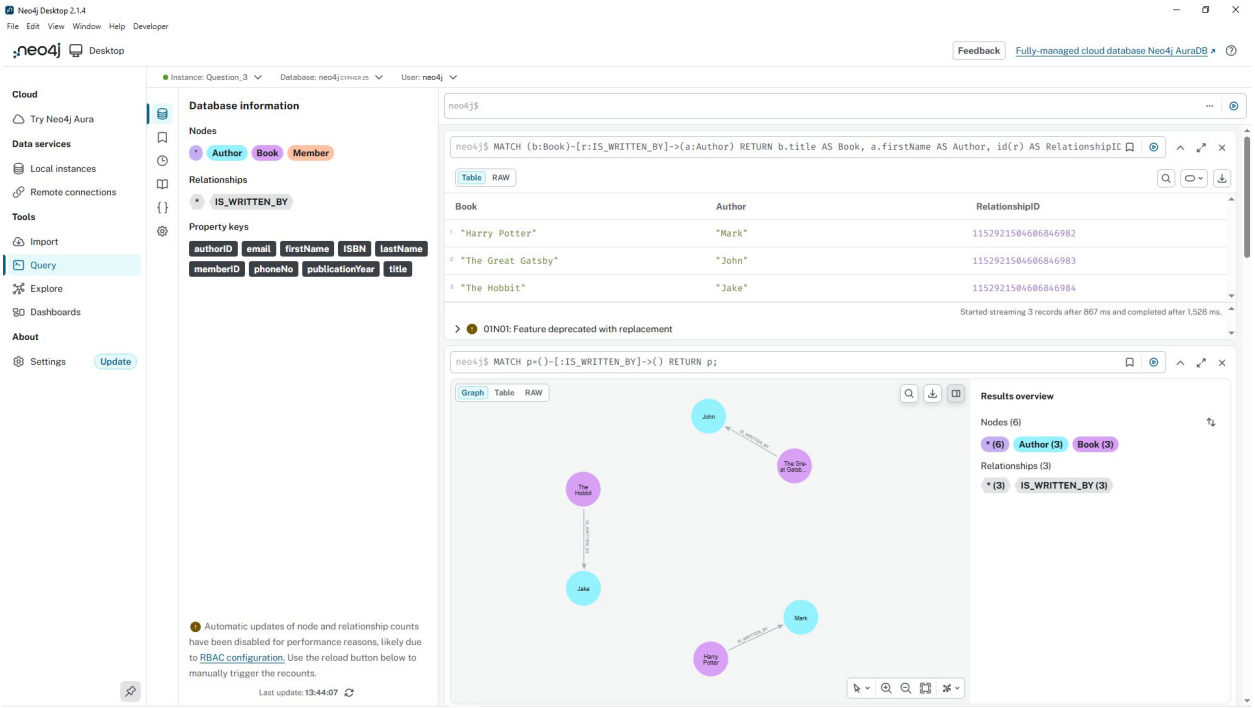
(i)

```
MATCH (b:Book {title: "Harry Potter"}), (a:Author {authorID: "au001"})
CREATE (b)-[:IS_WRITTEN_BY]->(a); MATCH (b:Book {title: "The Great
Gatsby"}), (a:Author {authorID: "au002"})
CREATE (b)-[:IS_WRITTEN_BY]->(a); MATCH (b:Book {title: "The Hobbit"}),
(a:Author {authorID: "au003"})
CREATE (b)-[:IS_WRITTEN_BY]->(a);

MATCH p=()-[:IS_WRITTEN_BY]->() RETURN p;
```




```
MATCH (b:Book)-[r:IS_WRITTEN_BY]->(a:Author)
RETURN b.title AS Book, a.firstName AS Author, id(r) AS RelationshipID;
```

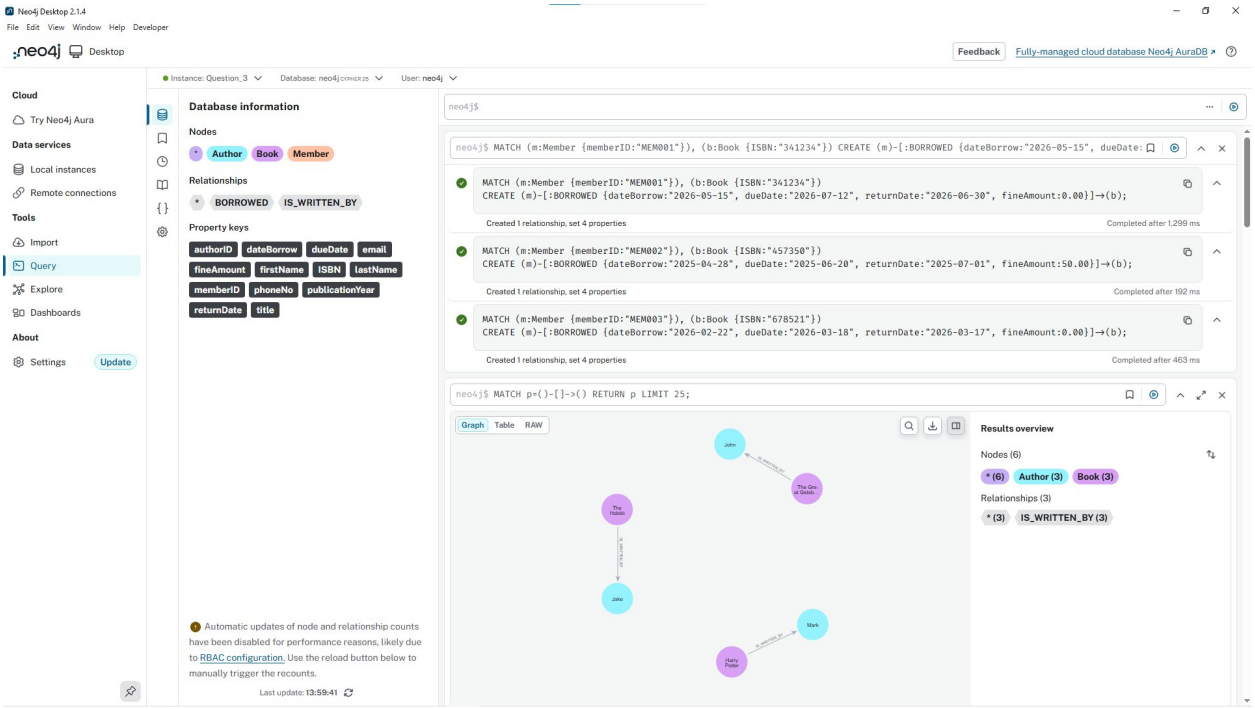


(ii)

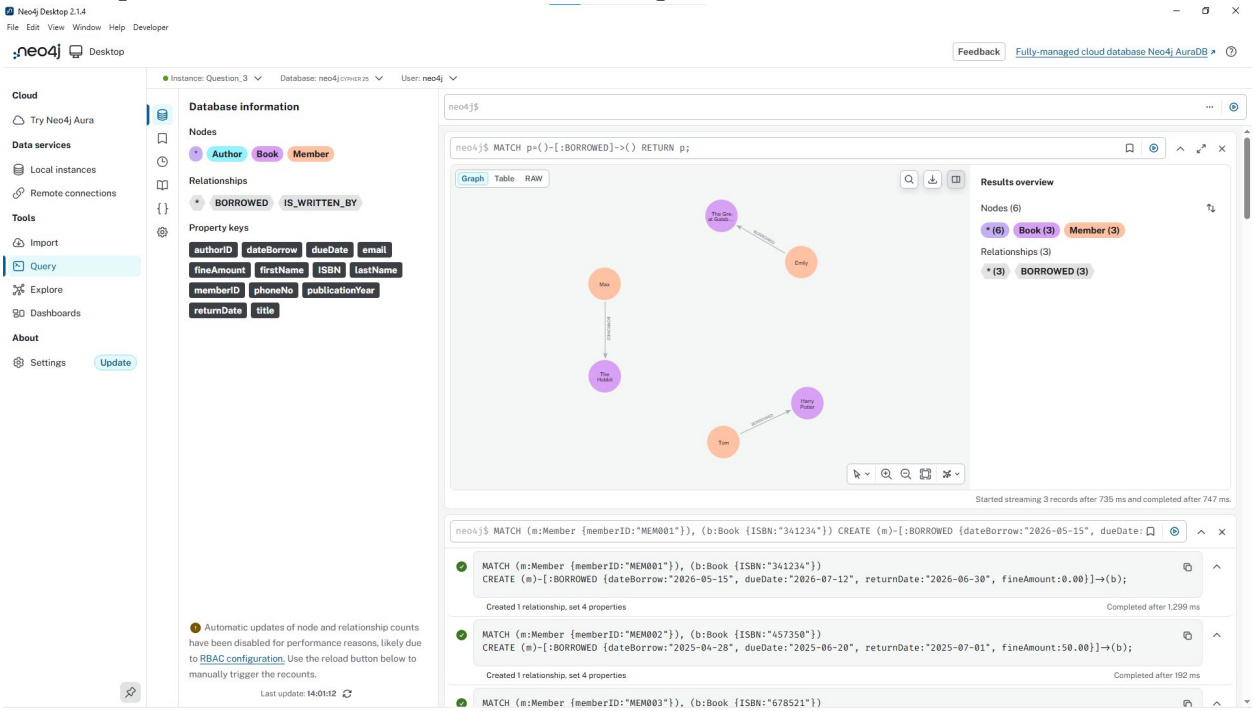
```
MATCH (m:Member {memberID:"MEM001"}), (b:Book {ISBN:"341234"})
CREATE (m)-[:BORROWED {dateBorrow:"2026-05-15", dueDate:"2026-07-12",
returnDate:"2026-06-30", fineAmount:0.00}]->(b);

MATCH (m:Member {memberID:"MEM002"}), (b:Book {ISBN:"457350"})
CREATE (m)-[:BORROWED {dateBorrow:"2025-04-28", dueDate:"2025-06-20",
returnDate:"2025-07-01", fineAmount:50.00}]->(b);
```

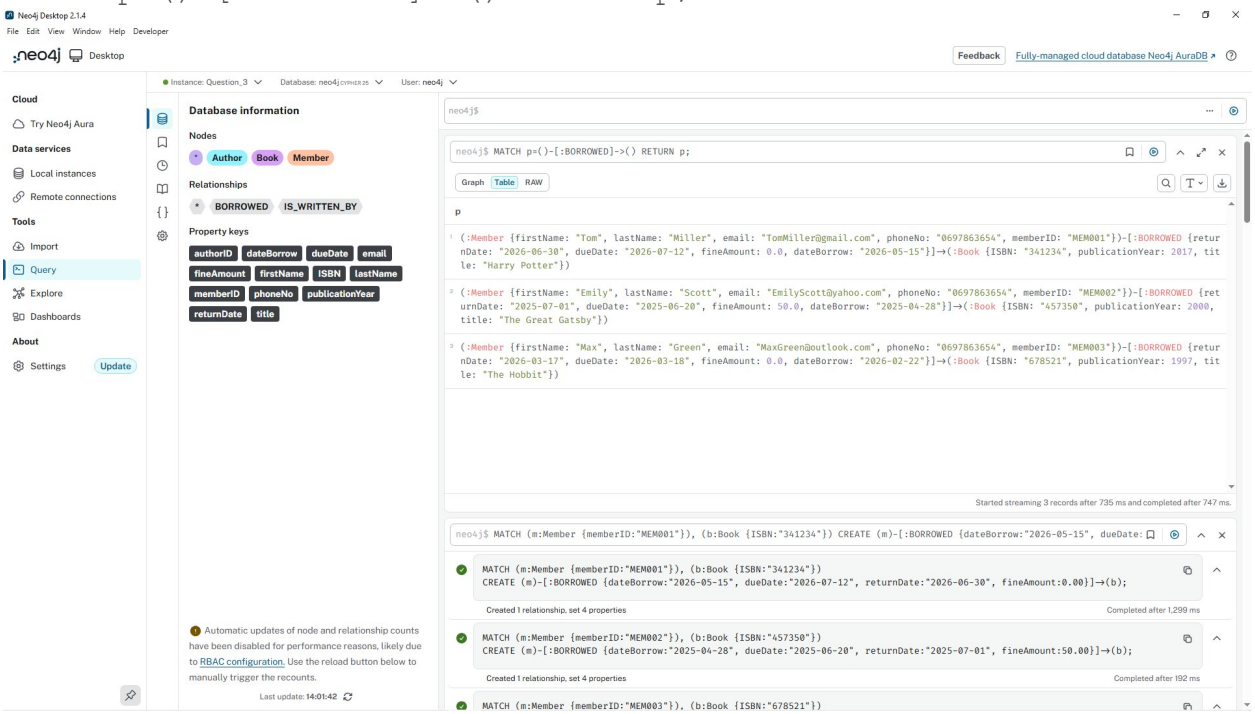
```
MATCH (m:Member {memberID:"MEM003"}), (b:Book {ISBN:"678521"})
CREATE (m)-[:BORROWED {dateBorrow:"2026-02-22", dueDate:"2026-03-18",
returnDate:"2026-03-17", fineAmount:0.00}]->(b);
```




```
MATCH p=()-[:BORROWED]->() RETURN p;
```



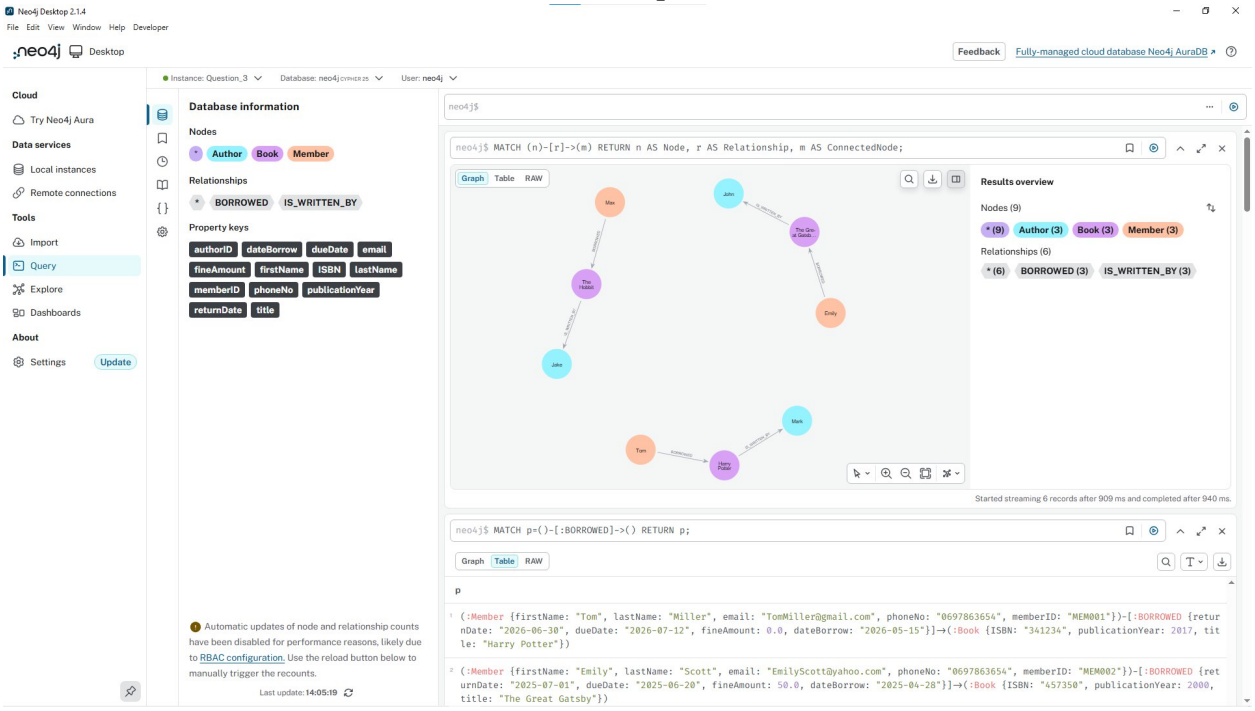
```
MATCH p=()-[:BORROWED]->() RETURN p;
```



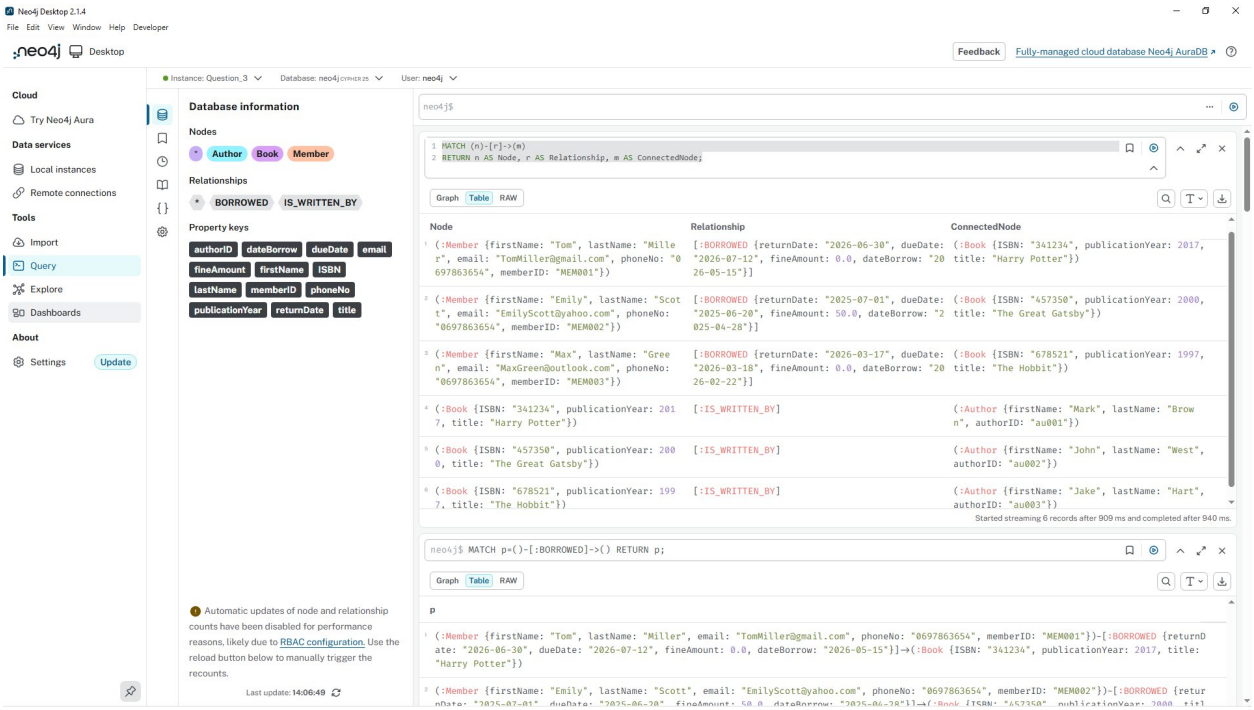
3.3)

(i)

```
MATCH (n)-[r]->(m)
RETURN n AS Node, r AS Relationship, m AS ConnectedNode;
```

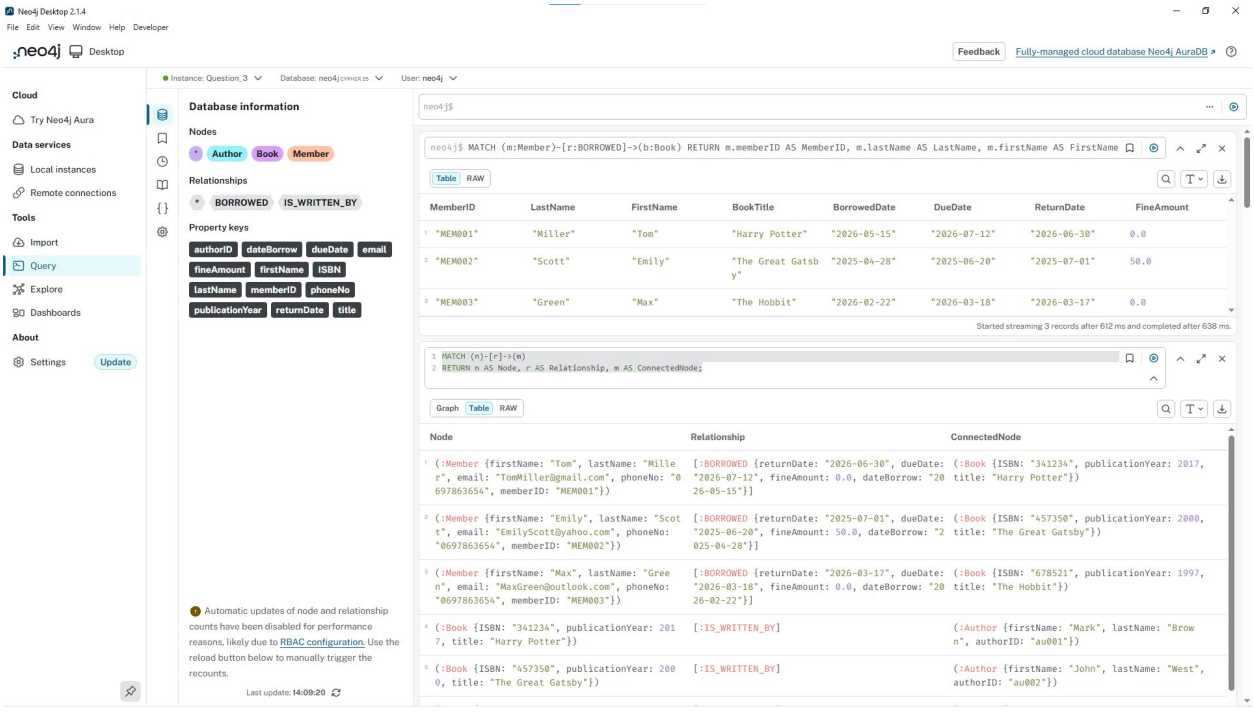


(ii)



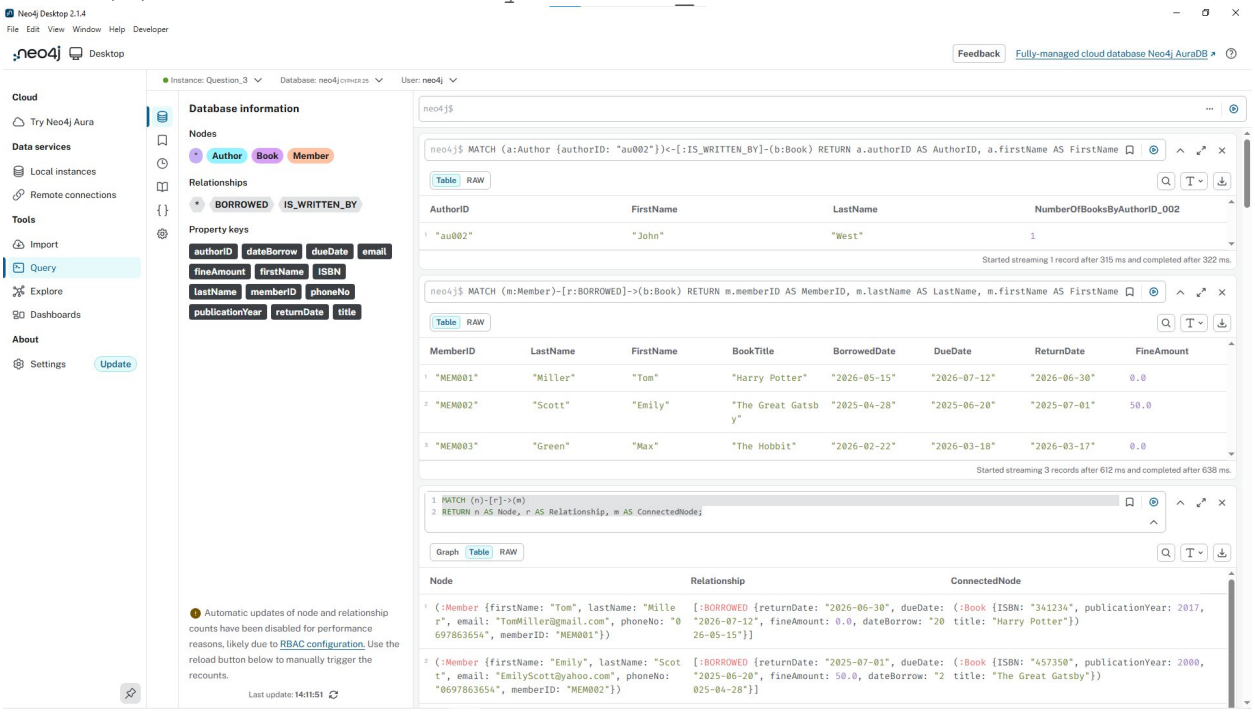
3.4)

```
MATCH (m:Member)-[r:BORROWED]->(b:Book)
RETURN
m.memberID AS MemberID,
m.lastName AS LastName,
m.firstName AS FirstName,
b.title AS BookTitle,
r.dateBorrow AS BorrowedDate,
r.dueDate AS DueDate,
r.returnDate AS ReturnDate,
r.fineAmount AS FineAmount;
```



3.5)

```
MATCH (a:Author {authorID: "au002"})<-[:IS_WRITTEN_BY]-(b:Book)
RETURN
a.authorID AS AuthorID,
a.firstName AS FirstName,
a.lastName AS LastName,
COUNT(b) AS NumberOfBooksByAuthorID_002;
```



3.6)

```
MATCH (m:Member)-[r:BORROWED]->(b:Book)
RETURN
m.memberID AS MemberIdentity,
b.title AS BookHasNotBeenReturn,
b.ISBN AS ISBN;
```

